Teil I

Objektorientierte Programmierung

0.1 Einleitung

0.1.1 Literatur

0.1.1.1 Literatur (Auswahl)

- Ulla Kirch, Peter Prinz.
  C++ Lernen und professionell anwenden
  mitp Verlag, 7. Auflage, 2015, *Gutes Lehrbuch*
- Bjarne Stroustrup.
  Die C++ Programmiersprache, Hanser, 2015,
  Referenz, sehr umfangreich
- Internetsuche nach C++ Befehlen, Fehlermeldungen, Beispielen, Tutorials
  `www.cplusplus.com/`, `de.cppreference.com/`, `http://www.learncpp.com/`
- Scott Meyers, Effektiv C++ programmieren, Addison-Wesley 2011
- Scott Meyers, Effektives modernes C++, Addison-Wesley 2015

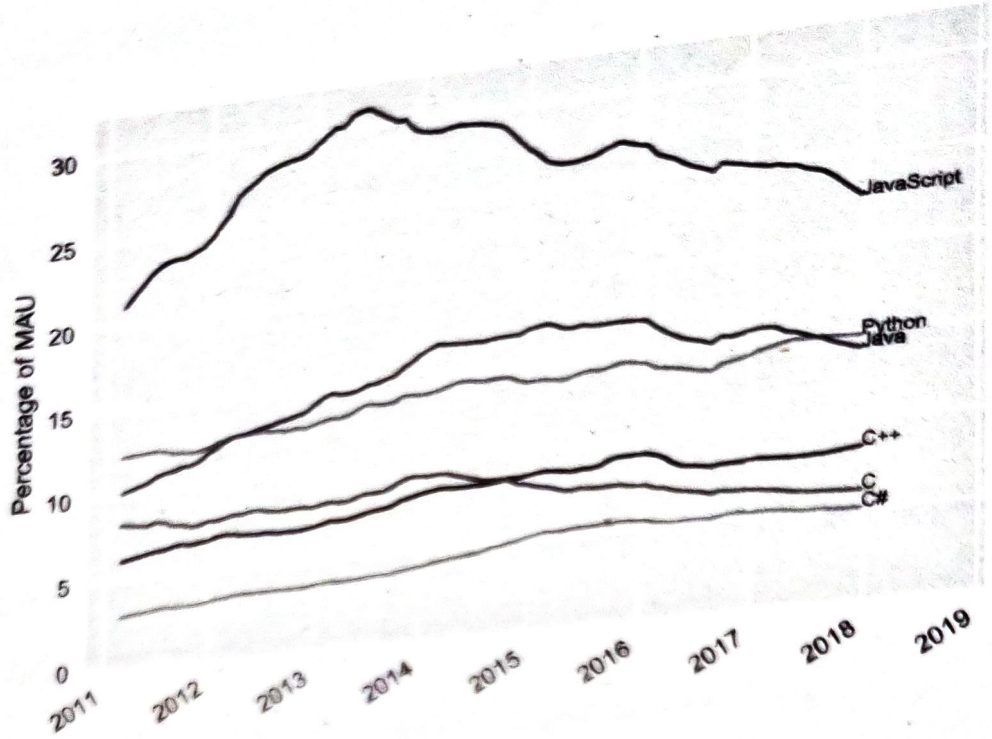
0.1.2 Ranking der Programmiersprachen

0.1.2.1 Quellen zum Ranking von Programmiersprachen

- `en.wikipedia.org/wiki/Measuring_programming_language_popularity`
- `de.wikipedia.org/wiki/TIOBE-Index`
- `spectrum.ieee.org/computing/software/the-2017-top-programming-languages`
- `redmonk.com/sogrady/2017/06/08/language-rankings-6-17/`
- `stackify.com/popular-programming-languages-2018/`
- `pypl.github.io/PYPL.html?country=DE`
- `www.benfrederickson.com/ranking-programming-languages-by-github-users/`
- `stackify.com/popular-programming-languages-2018/`
- `blog.newrelic.com/2017/10/09/popular-programming-languages-2017/`
- `www.levenez.com/lang/`

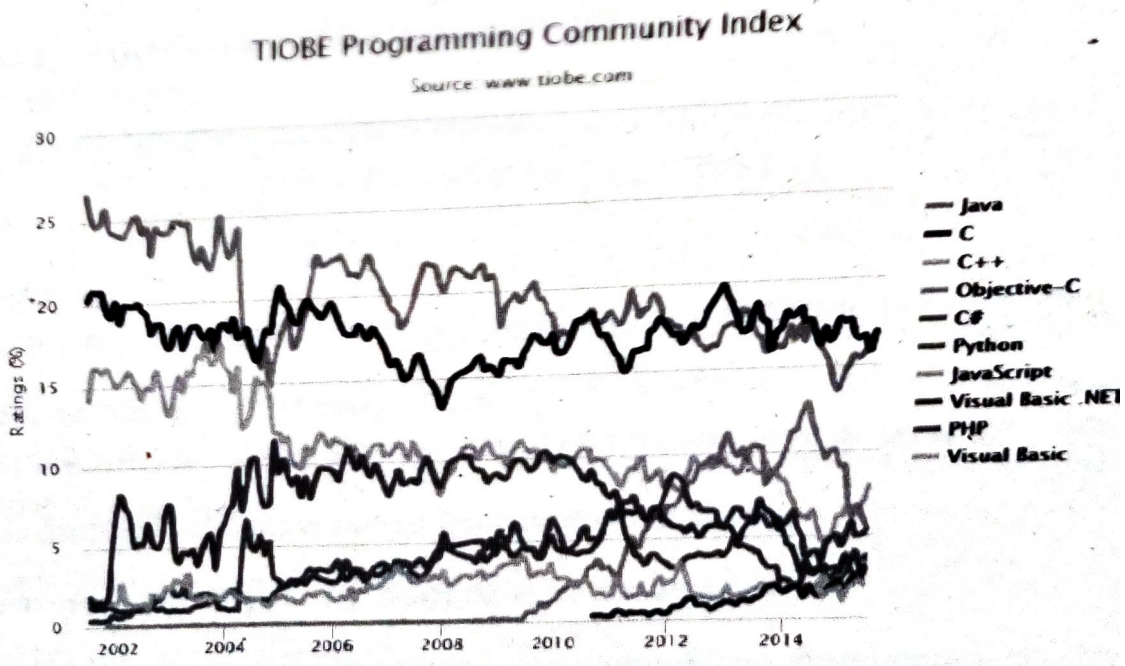
© R.-R. Grigat 2003-2019

0.1.2.2 Historie und Beliebtheit wichtiger Programmiersprachen



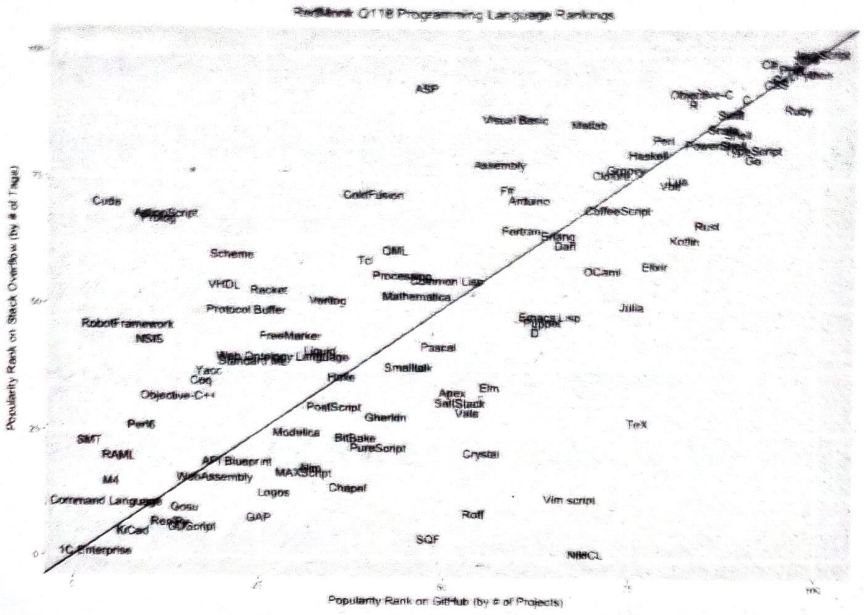
Source: www.benfrederickson.com/ranking-programming-languages-by-github-users/

0.1.2.3 Historie und Beliebtheit wichtiger Programmiersprachen



© R.-R. Grigat 2003-2019

0.1.2.4 Historie und Beliebtheit wichtiger Programmiersprachen



0.1.2.5 IEEE Ranking of programming languages

	Ranking				
Stand 2018	1.	2.	3.	4.	5.
Tiobe 2018	*Java*	C	*C++*	Python	C#
IEEE Jobs	*Java*	C	Python	*C++*	JavaScript
IEEE Trend	Python	C	*C++*	*Java*	Swift
IEEE Open	Python	*C++*	*Java*	C	Swift
PYPL	Python	*Java*	C#	PHP	JavaScript
GitHub Pulls	JavaScript	Python	*Java*	*C++*	C
Stackify	*Java*	C	*C++*	Python	C#
RedMonk	JavaScript	*Java*	Python	PHP	C#

0.1.3 Tipps für C-Programmierer und C++-Programmierer

0.1.3.1 Tipps für C-Programmierer

Tipps für C-Programmierer (Stroustrup)

- Schreiben Sie in C++ kein C, nutzen Sie die C++ Vorteile, insbesondere die C++ Standardbibliothek STL.
- Beachten Sie Unterschiede zur C Standardbibliothek, z.B. = statt strcpy() und == statt strcmp()
- Statt Makros verwenden Sie **`const`**, `constexpr`, **`enum`**, **`enum class`**, **`inline`**, Templates, Namespaces.

© R.-R. Grigat 2003-2019

- Deklarieren Sie Variablen erst bei Verwendung und initialisieren Sie sie sofort.
- Stets **new** statt `malloc()`, `vector` statt `realloc()`.
- **new/delete** möglichst nur in Konstruktor/Destruktor
- Vermeiden Sie void*, Unions und Typumwandlungen. Notfalls verwenden Sie **static_cast** und **dynamic_cast**.
- Verwenden Sie `string`, `array`, `vector` der Standardbibliothek statt Array und C-Strings.

0.1.3.2 Tipps für C++-Programmierer

Tipps für C++-Programmierer (Stroustrup)

- Verwenden Sie Konstruktoren um Invarianten einzurichten.
- Container und Algorithmen statt Arrays und ad hoc Code.
- Verwenden Sie STL und Exceptions.
- Move-Semantik statt Kopien großer Objekte
- `unique_ptr` zum Verweis auf polymorphe Objekte
- `shared_ptr` zum Verweis auf gemeinsame Objekte
- Templates eliminieren Typumwandlungen und vereinfachen Klassenhierarchien.

© R.-R. Grigat 2003-2019

0.2 Imperative (prozedurale) Sprachen

0.2.1 Grundlagen

0.2.1.1 Typ, Deklaration, Variable

Grundbegriffe

Typ definiert zulässige *Werte und Operationen*. Jeder Name und jeder Ausdruck besitzt einen Typ.

Deklaration führt einen Namen ins Programm ein und spezifiziert einen Typ für die benannte Entität.

Objekt ist etwas im Hauptspeicher, das einen Wert eines bestimmten Typs speichert.

Wert ist Menge von Bits, die entsprechend dem Typ interpretiert werden.

Variable ist ein benanntes Objekt.

0.2.1.2 Initialisierungliste und auto

Initialisierungsliste und auto

```
1 int i1 =  7.2 ;  // i1 wird zu 7
2 int i2   {7.2};  // Fehler, Umwandlung von Gleitkommazahl in Ganzzahl
3 int i2 = {7.2};  // Derselbe Fehler, das = ist redundant
4 int i3[] = {2,4,6} // Array initialisieren mit initializer_list<int>
5 auto b = true;   // Typ ist bool
6 auto ch = 'x';   // Typ ist char
7 auto i = 1;      // Typ ist integer
8 auto j {1};      // Aber: Typ ist Liste initializer_list<int>
9 auto d  = 1.2;   // Typ ist double
```

- Einschränkende Konvertierungen werden durch Initialisierungslisten `initializer_list<T>` unterbunden
- Konstanten müssen initialisiert werden
- Variablen sollten initialisiert werden
- Bei **auto** Syntax mit =, da keine Typumwandlung stattfindet
- **auto** `i=3;` ist integer, aber **auto** `i{3};` ist *Liste* von int
- **auto** ist besonders hilfreich bei langen Typnamen

© R.-R. Grigat 2003-2019

0.2.1.3 Globale Variable vs. lokale Variable

Definition: Globale Variable

1. Eine Variable, die außerhalb jeder Funktion definiert wird, heißt *globale Variable*.
2. Auf eine globale Variable kann während der gesamten Laufzeit des Programms zugegriffen werden.

Definition: Lokale Variable
Eine *lokale Variable* existiert jeweils nur in ihrem mit {} eingeschlossenen Gültigkeitsbereich.

Hinweis:

- Lokale Variablen verwenden, falls möglich.
- Ausnahme z. B. Konstanten, globale Einstellungen
- Rückgabe lokaler Variablen aus Funktion nur als Kopie

0.2.1.4 Zuweisung vs. Initialisierung

Definition: Zuweisung
Zuweisung ist Überschreiben einer Variable mit Wert.

Definition: Initialisierung
Initialisierung ist das *erstmalige* Beschreiben einer Variablen mit einem Wert zum Zeitpunkt der Zuordnung des Speicherbereichs zu dieser Variable.

Achtung:

- In C++ werden Variablen i. A. *nicht* automatisch initialisiert, d. h. die Belegung von Variablen ist ohne Initialisierung durch den Programmierer zufällig.
- C++ Compiler überprüft i. A. nicht, ob Variable vor Auslesen initialisiert wurde (Unterschied zu Java)
- Fehlende Initialisierung ist häufige Fehlerursache in C++

$\Rightarrow$ Variablen immer initialisieren!

© R.-R. Grigat 2003-2019

0.2.1.5 const und volatile

const und volatile

- **const** Variablen müssen initialisiert werden, sind danach unveränderlich.
- **volatile** Variablen können von außerhalb des Programms geändert werden (z.B. durch die Systemuhr)
- Variablen, die **const** und **volatile** sind können nicht vom Programm, aber von außerhalb geändert werden.

```
volatile const unsigned long clock_ticks;
```

0.2.1.6 For-Schleifen

For-Schleifen

```
int v[] = {0,1,2,3,4,5,6,7,8,9};

for (auto i=0; i!=10; ++i)
   std::cout << v[i] << ", ";

for (auto x: v)   // x ist Kopie der Elemente aus v
   std::cout << x << ", ";

for (auto& x: v)  // x ist keine Kopie sondern Referenz
   ++x;
```

0.2.1.7 C++-Strings und C-Strings

C++-Strings und C-Strings

- Befehlsvorrat der C++-Stringklasse `std::basic_string` mit Header <string> ist mächtiger als bei C-Strings. API: `http://en.cppreference.com/w/cpp/string/basic_string`
- C-Strings (Null-terminierte Byte-Strings) mit Header <cstring> sind durch '\0' terminierte char-Arrays. API: `http://en.cppreference.com/w/cpp/string/byte`

© R.-R. Grigat 2003-2019

- String-Literal "Hallo" ist Konstante vom Typ const char[6] mit 6 Zeichen einschließlich '\0'.
  - Operatoren von <string> funktionieren daher *nicht* für konstante Stringliterale, z.B. "Hallo"+"Hallo".
  - Für C++-String s ist s+"Hallo"+"Hallo" dagegen erlaubt.

```
1 const char p1[6] = "Hallo";  // Konstantes Stringliteral
2 char p2[] = "Hallo";         // Ok, Cast auf veränderlichen String
3 char* p3 = "Hallo"; // Warnung, kein ISO C++, Längeninfo nur durch \0
4 const char* p4 = "Hallo";    // Ok
```

0.2.1.8 Gegenüberstellung C++- und C-Strings

Gegenüberstellung C++- und C-Strings

Erläuterung	C++-Strings	C-Strings
Typ	string	char*
Headerdatei	<string>	<cstring>
Terminierung	keine	Zeichen \0
Vergleich	==	strcmp()
Zeichen zählen	size(),length()	strlen()
Zuweisung	=	strcpy()
String nach double	stod()	atof()
String nach int	stoi()	atoi()
Suchen	find()	strstr()
Adresse drucken	&myString	(void*)myString
Zeile einlesen	getline()	getchar()

Beispiele C++-Strings: KP09_1_string_getline, KP09_2_length_summation, KP09_3_kleiner_sync, KP09_4_isspace_cctype, KP16_4_array_classKonto, KP16_5_multidimArrays, KP16_6_listeAlsArray_Telefonliste, KP17_1_textPtr, KP17_3_kleiner_sync

Beispiele C-Strings: KP16_3_C-String, KP17_1_textPtr, KP17_4_reverse, KP17_5_suche_strstrEigeneVersion

0.2.1.9 Eingabe von Einzelzeichen durch Tastatur

Verarbeitung von Einzelzeichen

- In Eclipse Auswertung von Tastaturdaten nach RETURN.

© R.-R. Grigat 2003-2019

  - `getchar()`, Header <cstdio>, liefert sequentiell alle Zeichen samt RETURN.
  - `getline()`, Header <string>, liefert STL-String oder C-String ohne RETURN.

- Header <conio.h> ist nicht konform zum Standard.
- `getch()` aus <conio.h> funktioniert nicht in Eclipse.

```
#include <iostream>
#include <string>
int main() {
    std::string name;
    std::cout << "Bitte Namen und RETURN eingeben: ";
    std::getline(std::cin, name);
    for(auto& c : name)
        if (c!='e') putchar(c);  // 'e' unterdrücken
}
```

0.2.1.10 Zeitmessung

Zeitmessung

- Zeiten, Zeiträume, Zeitpunkte in <chrono> im (Teil-)Namespace `std::chrono`.
- Zeitpunkt: `std::chrono::time_point` entspricht Zählerstand der gewählten Uhr.
- Zeitintervall: `std::chrono::duration`
  - Anzahl Zeitschritte vom Typ Rep (für Repräsentation)
  - Zeiteinheiten vom Typ `std::ratio` (Schrittweite)
  - getter-Methode `count` für Anzahl der Zeitschritte
- Drei Zeitgeber stehen zur Auswahl
  - `system_clock` liefert Zeit der System-Echtzeituhr
  - `steady_clock` ist monotone Uhr ohne Anpassung
  - `high_resolution_clock` hat kürzestmögliche Ticks

© R.-R. Grigat 2003-2019

0.2.1.11 Zeiteinheiten

Zeiteinheiten

- Beispiele für duration<Rep,Period>

```
1 duration<long long,milli> d1 {7}; // 7 Millisekunden
2 duration<double,pico> d2 {3.33};  // 3,33 Pikosekunden
3 duration<int,ratio<1,1>> d3 {};   // 0 Sekunden
```

- Zuweisung nur mit konsistenten Einheiten

```
1 duration<long long,milli> d1 {7}; // 7 Millisekunden
2 d1 += 5; // Fehler, Einheit von 5 undefiniert
3 duration<int,ratio<1,1>> d2 {7}; // 7 Sekunden
4 d1 += duration<long long,milli>{5}; // OK: Millisekunden
5 d2 += decltype(d2){5}; // OK: Sekunden
6 d2 = duration_cast<milli>(d2); // Umrechnung in Millisekunden
```

0.2.1.12 Vordefinierte Zeiteinheiten

Vordefinierte Zeiteinheiten

- Vordefinierte SI-Einheiten der Standardbibliothek.
- siN ist ein "implementierungsabhängiger vorzeichenbehafteter Ganzzahltyp mit mindestens N Bits".

```
1 using nanoseconds = duration<si64,nano>;
2 using microseconds = duration<si55,micro>;
3 using milliseconds = duration<si45,milli>;
4 using seconds = duration<si35>;
5 using minutes = duration<si29,ratio<60>>;
6 using hours = duration<si23,ratio<3600>>;
```

- Datum und Uhrzeit kann auch im C-Stil gemessen werden mit <ctime>, wie std::time_t, std::difftime, CLOCKS_PER_SEC

© R.-R. Grigat 2003-2019

0.2.1.13 Einfaches Beispiel für chrono

Einfache Beispiele für chrono

- Zeitmessung mit `steady_clock`

```
steady_clock::time_point t = steady_clock::now();
// ... etwas tun ...
steady_clock::duration d = steady_clock::now()-t; // d Zeiteinheiten
```

- Kürzer formuliert mittels **auto**

```
auto t = steady_clock::now();
// ... etwas tun ...
auto d = steady_clock::now()-t; // d Zeiteinheiten verbraucht
// ausgeben als Millisekunden:
cout << "Rechenzeit " << duration_cast<milliseconds>(d).count() << "ms";
```

0.2.1.14 Beispiel Zeitmessung

Zeitmessung mit <chrono> und <ctime>

```
#include <iostream>
#include <chrono>
#include <ctime>

int fibonacci(int n){
    if (n < 3) return 1;
    return fibonacci(n-1) + fibonacci(n-2);
}
int main(){
    std::chrono::time_point<std::chrono::system_clock> start, end;
    start = std::chrono::system_clock::now();
    int result = fibonacci(42);
    end = std::chrono::system_clock::now();
    int elapsed_seconds = std::chrono::duration_cast<std::chrono::seconds>
                          (end-start).count();
    std::time_t end_time = std::chrono::system_clock::to_time_t(end);
    std::cout << "finished computation at " << std::ctime(&end_time)
              << "elapsed time: " << elapsed_seconds << "s\n";
}
```

Beispiel: Zeitmessung_chrono_fibonacci, Zeitmessung_ctime

© R.-R. Grigat 2003-2019

0.2.2 Mehr über Zeiger

0.2.2.1 Zeiger auf Zeiger

Zeiger auf Zeiger

```
1 CAuto** ptr = new CAuto*[400] // Array aus Zeigern auf Autos
```

- `ptr` adressiert ersten Zeiger im Array
- Arrayelement *i* ansprechen `*ptr` und `ptr[0]` Zeiger bei Index 0 `*(ptr+i)` und `ptr[i]` Zeiger bei Index i
- Zugriff auf die vom Array verwalteten Objekte `**ptr` und `*ptr[0]` Objekt, das der 0-te Zeiger adressiert `**(ptr+i)` und `*ptr[i]` Objekt, das der `i`-te Zeiger adressiert (`i`=0,1,2,...)

Beispiel: KP12_5_wertUndAdresseVonVariablen, KP30_1_zeigerAufZeiger

0.2.2.2 Variable Anzahl von Funktionsargumenten

Variable Anzahl der Aufrufparameter

- Variable Anzahl *optionaler* Aufrufparameter zulässig
- Zugriff durch *Argument-Zeiger* vom Typ `va_list`
- Davor mindestens ein *obligatorischer* Aufrufparameter

```
1 #include <cstdarg>
2 int f(int max, ...);  // max obligatorisch, ... optionale Parameter
```

- `va_start(argPtr, lastObligArg)` setzt Argument-Zeiger `argPtr` auf erstes optionales Argument hinter letztem obligatorischen Parameter `lastObligArg`.
- `va_arg(argPtr, argType)` liest optionales Argument vom Typ `argType` mittels `argPtr` und inkrementiert `argPtr`.
- Nach Ende der Auswertung wird `argPtr` auf `nullptr` gesetzt durch `va_end(argPtr);`
- Für mehrfaches Lesen mit `va_start()` neu beginnen

Beispiel: KP30_2_variableAnzahlArgs

© R.-R. Grigat 2003-2019

0.2.2.3 Zeiger auf Funktionen

Zeiger auf Funktionen (Funktionszeiger)

- Name einer Funktion ist *konstanter* Zeiger auf die Funktion
- Zeiger adressiert Maschinencode der Funktion
- Anwendungsbeispiel Sprungtabelle (vtable)
- Mit einem *Funktionszeiger* (Zeiger auf Funktion) kann man die Adresse der Funktion abrufen und sie aufrufen, nicht aber die Funktion ändern.
- **`double(int)`** ist der Typ einer *Funktion* mit Aufrufparameter **`int`** und Rückgabetyp **`double`**.
- **`double(*)(int)`** ist Typ eines *Zeigers* auf eine Funktion.
- **`double(*pf)(int)`** ist ein benannter Zeigertyp.
- **`double *pf (int)`** ist dagegen Funktion `pf` mit Rückgabetyp **`double*`**.

0.2.2.4 Beispiel Zeiger auf Funktionen

```
1  #include <iostream>
2  #include <string>
3  void drucke(std::string s){std::cout << s << std::endl;} // void(string)
4  void drucke(int  i){std::cout << i << std::endl;}        // void(int)
5
6  int main(){
7     void (*f1)(std::string) = &drucke; // f1 zeigt auf drucke
8     void (*f2)(std::string) = drucke;  // auch ok, implizites &
9     void (*f3)(std::string);  // Zeiger auf void(string)
10    f3 = &drucke;             // f3 zeigt nun auf drucke void(string)
11    f3 = drucke;              // auch ok
12    void (*f4)(int) = drucke; // f4 zeigt nun auf drucke void(int)
13
14    (*f1)("Erster Test mit f1"); // Aufruf von drucken
15    f1("Zweiter Test mit f1");   // Auch ok, Dereferenzierung automatisch
16    f2("Test mit f2");
17    f3("Test mit f3");
18 // f1(42);         // Fehler, falscher Argumenttyp
19    f4(42);         // ok
20    return 0;
21 }
```

© R.-R. Grigat 2003-2019

Beispiel: Funktionszeiger, KP30_3_sprungtabelleFunktionen

0.2.2.5 Komplexe Deklarationen

Komplexe Deklarationen

- Komplexe Deklaration einer Funktion ist eine Deklaration mit mehr als einem Operator aus der Menge von Indexoperator [], Funktion (), Zeiger *, Referenz &.
  - Elemente eines Arrays können keine Funktionen sein.
  - Eine Funktion kann keine Funktion und kein Array zurückgeben, wohl aber Zeiger darauf.
  - Klammersetzung ist zulässig
  - Rechte Operatoren (), [] binden stärker als linke *, &
- Regeln zum Parsen komplexer Deklarationen
  1. Man beginnt beim Bezeichner, der deklariert wird.
  2. Steht rechts () oder [], dann wird dieses interpretiert.
  3. Steht rechts nichts oder), wird links der Stern interpretiert, falls vorhanden.
  4. Wiederhole die vorherigen zwei Schritte, bis alle Operatoren aufgelöst sind.
  5. Zuletzt wird der Grundtyp angewendet.

0.2.2.6 Beispiele für komplexe Deklarationen

Beispiele für komplexe Deklarationen

```
1 char  ( *  strptr)[50]
2   3.    1.   0.    2.
```

$\overset{0}{\rightarrow}$ `strptr` ist $\overset{1}{\rightarrow}$ Zeiger auf $\overset{2}{\rightarrow}$ Array mit 50 Elementen $\overset{3}{\rightarrow}$ vom Typ **char**.

```
1 long * (*  func () ) []
2   5. 4. 2.   0. 1.   3.
```

$\overset{0}{\rightarrow}$ `func` ist $\overset{1}{\rightarrow}$ Funktion $\overset{2}{\rightarrow}$ mit Return-Wert vom Typ Zeiger $\overset{3}{\rightarrow}$ auf Array $\overset{4}{\rightarrow}$ mit Elementen vom Typ Zeiger $\overset{5}{\rightarrow}$ auf **long**.

```
1 char * (* (* funcptr) () ) []
2 6.   5. 3. 1.   0.    2.   4.
```

$\overset{0}{\rightarrow}$ `funcptr` ist $\overset{1}{\rightarrow}$ Zeiger $\overset{2}{\rightarrow}$ auf Funktion $\overset{3}{\rightarrow}$ mit Return-Wert vom Typ Zeiger $\overset{4}{\rightarrow}$ auf Array $\overset{5}{\rightarrow}$ mit Elementen vom Typ Zeiger $\overset{6}{\rightarrow}$ auf **char**.

© R.-R. Grigat 2003-2019

0.2.2.7 Typalias: typedef versus using

Typalias: typedef versus using

- **using** (moderner) und **typedef** (traditionell)
- *Neuer Name* für bereits existierenden Datentyp
- Portabilität durch Kapselung maschinenabhängiger Typen
- Typalias Namenskonvention: `complex_t` oder `COMPLEX`

```
1 using COMPLEX = struct{double re, im;};  // komplexe Zahl
2 typedef struct{double re, im;} COMPLEX;  // dasselbe
3 COMPLEX z1, z2, *pz; // zwei komplexe Zahlen, Zeiger auf komplexe Zahl
```

```
1 using WERKTAG = enum{Mo, Di, Mi, Do, Fr};  WERKTAG tag;
```

```
1 using PTR_TO_FUNC = char*(*)(); // Zeiger auf Funktion, die char* liefert
2 typedef char*(*PTR_TO_FUNC)();  // dasselbe
3 PTR_TO_FUNC search;  // gleichbedeutend mit char*(*search)();
```

```
1 template<typename T> using Vector = std::vector<T, My_allocator<T>>;
2 Vector<int>  entfernungen;  // template-Alias
```

0.2.2.8 Matrizen als Argumente von Funktionen

Matrizen als Argumente von Funktionen

- Spaltenvektoren (Arrays) aus Zeilenvektoren (Arrays).
- Name eines Arrays ist Zeiger auf erstes Arrayelement.
- Name einer Matrix ist Zeiger auf ersten Zeilenvektor.
- Compiler muss für Typ einer Matrix die Zahl der Spalten und Zeilen kennen.
- `size_t` ist Ganzzahltyp für die Größe von Arrays.

© R.-R. Grigat 2003-2019

```
1 const size_t COLS{3};
2 const size_t ROWS{2};
3 using Row_t = int[COLS];  // äquivalent:  typedef int Row_t[COLS];
4 using Matrix_t = int[ROWS][COLS];  // typedef int Matrix_t[ROWS][COLS];
5 Row_t *pMat;     // äquivalent:  Row_t pMat[];
6 Row_t mat[5];  // Matrix aus 5 Zeilen und 3 Spalten
7 Matrix_t m;  // Matrix vom Typ Matrix_t, 2 Zeilen, 3 Spalten
8 m[1][2] = 7; // Zweite Zeile, dritte Spalte
```

Beispiel: MatrixInCppArray, KP16_5_multidimArrays, KP30_4_dynamischeMatrizen

0.2.3 Unterprogramme

- Lösungsansatz für komplexe Probleme: Zerlegung in Teilprobleme
  - beim Softwareengineering *Top-Down-Entwurf* genannt
- imperative Programmiersprachen realisieren diese Zerlegung durch *Unterprogramme*
- in manchen Sprachen (z. B. Pascal) Unterscheidung zwischen

  Prozedur Verbundanweisung zur Veränderung von Daten

  Funktion Verbundanweisung, die einen Wert zurückliefert und ggf. ebenfalls Daten ändert

0.2.3.1 Funktionen in C++ / Java

Beispiel einer Funktionsdefinition

```
1 long int fakultaet(int x)
2 {                          // Beginn Anweisungsblock
3   long int f = 1;
4   for(int i = x; i > 1; i--)
5   {
6     f = f * i;
7   }
8   return f;
9 }                          // Ende Anweisungsblock
```

© R.-R. Grigat 2003-2019

Funktionsdefinitionen bestehen aus

- (Rückgabe-)Typ (`long int`)
- Funktionsname (`fakultaet`)
- Aufrufparameter (`int x`)
- Anweisungsblock (in {} eingeschlossen)

0.2.3.2 Deklaration mittels Prototyp

- Jeder Name (Bezeichner) muss vor Verwendung (z. B. im Anweisungsblock einer Funktion) bekannt sein
- Problem bei *rekursiven* Definitionen (Funktion ruft sich selbst auf, müsste dazu bei Definition schon definiert sein)
- Abhilfe: *Deklaration mittels Prototyp* vorab. Ein Prototyp verbindet einen Namen mit Typen.

Funktionsdeklarationen bestehen aus

- Rückgabetyp (`long int`)
- Funktionsname (`fakultaet`)
- Parameterliste (`int x`)
- Anweisungsblock *nur optional* (Prototyp stets *ohne* Anweisungsblock)

Beispiel für Deklaration (hier Prototyp)

```
long int fakultaet(int x);
```

0.2.3.3 Prototyp vs. Definition

Funktionsprototyp	Funktionsdefinitionen
Rückgabetyp (`long int`)	
Funktionsname (`fakultaet`)	
Parameterliste (`int x`)	
kein Anweisungsblock	Anweisungsblock

- Definition muss genau einmal erfolgen, Prototyp (d. h. Deklaration *ohne* Anweisungsblock) ist mehrfach erlaubt

© R.-R. Grigat 2003-2019

- Jede Definition ist auch Deklaration (aber nicht umgekehrt)
- Deklaration/Prototyp verbinden Namen mit Typen
- Definition legt für einen Namen eine *Entität* fest.
- Die Entität bezeichnet z. B. bei Funktionen deren Anweisungsblock, bei Klassen deren Daten und Methoden, bei Grundtypen deren allozierten Speicher.

0.2.3.4 Allgemeine Struktur von Deklarationen

Bestandteile der Deklaration

- Optionaler Spezifizierer, z.B. `static`, `virtual`
- Basistyp, z.B. `vector<double>` oder `const int`
- *Deklarator*, optional mit Namen, z.B. `p[7]`, `n`
- Optionaler Funktionsspezifizierer als Suffix, z.B. `const`, `noexcept`
- Optional ein Initialisierer oder ein Funktionsrumpf, z.B. `={7,5,3}` oder `{return x;}`
- Semikolon (außer bei Funktionen oder Namespaces)

0.2.3.5 Deklaratoroperatoren

Beispiele für Deklaratoroperatoren

Präfix	*	Zeiger
Präfix	*const	konstanter Zeiger
Präfix	*volatile	flüchtigerZeiger
Präfix	&	L-Wert-Referenz
Präfix	&&	R-Wert-Referenz
Präfix	auto	Funktion mit Suffixrückgabetyp
Postfix	[]	Array
Postfix	()	Funktion
Postfix	->	Rückkehr von Funktion

Postfix bindet stärker als Präfix!

```
1 int * feld[3];  // Array aus drei Zeigern, int* (feld[3]);
2 int (*feld)[3]; // Zeiger auf Array aus drei Integers.
```

© R.-R. Grigat 2003-2019

0.2.3.6 Hauptprogramm `main()`

- jedes C++ Programm benötigt genau eine `main()`-Funktion
- `main()` wird vom Betriebssystem aufgerufen
- muss (gemäß ANSI-Standard) einen **`int`**-Wert zurückliefern (bei vielen Betriebssystemen: $0 \equiv$ „kein Fehler“)

C++

```
#include<iostream>   // Funktionsdeklarationen fuer Ein- und Ausgabe
using namespace std; // Namensraum std
int main(int argc, char* argv[])
{
 cout << "Hello World!" << endl;
 return 0;  // Rückgabewert 0 == kein Fehler
}
```

0.2.4 Aufruf- und Rückgabeparameter

0.2.4.1 Parameterübergabe (Call by Value vs. Call by Reference)

Bei der Übergabe von Parametervariablen an eine Funktion unterscheidet man zwischen:

Call by Value

- Übergabe einer *Kopie* der Parametervariable
- Wert der ursprünglichen Parametervariablen kann durch Funktion *nicht* verändert werden

Call by Reference

- Funktion arbeitet auf Original-Parametervariable
- Wert der Parametervariable wird nicht kopiert
- Wert der Parametervariable kann verändert werden
- schneller als Call by Value, da keine Kopie erzeugt wird

Merke: Unterscheidung erforderlich zwischen

- allgemeinem Referenz-Begriff in „Call by Reference"
- Konzept der Referenzvariable „Referenz(C++)“ in C++

© R.-R. Grigat 2003-2019

0.2.4.2 Parameterübergabe: Call by Value

Beispiel C++

```
1 int verdoppeln(int x)
2 {
3     x = x * 2;
4     return x;
5 }

7 int a = 3;
8 int b = verdoppeln (a);
9 cout << a << ", " << b;
```

- *Kopie* von a an Funktion übergeben
- Rückgabewert der Funktion in b gespeichert
- a *nicht* verändert
- Ausgabe: 3, 6

Beispiel: CallByValRefPt, ReturnRefToLocalVar, KP12_1_referenzConstReferenz, KP12_2_callByReference, KP12_3_returnReferenz, KP12_6_swapperMitZeigern, KP14_5_returnReferenceToLocalVariable

0.2.4.3 Inhaltsoperator, Adressoperator, Referenz, Zeiger

& Adressoperator	y=&x; bedeutet: y ist die Adresse von x
int& Referenz(C++)	**int&** x = a; bedeutet: x ist Referenz(C++) auf **int** Variable a, x ist anderer *Name* für **int** Variable a, x ist der Speicher*inhalt*, nicht dessen Adresse Zuordnung von x wird *initialisiert*, Zuordnung ist *nicht* änderbar!
* Inhaltsoperator	*y ist der Inhalt von Adresse y, auch Dereferenzierung der Adresse y genannt
int* Zeigertyp	**int** * y bedeutet: y ist (änderbarer) Zeiger auf **int** Variable, y können andere Adressen zugewiesen werden

Tabelle 1: Die Bedeutung von &, **int&**, *, **int***

© R.-R. Grigat 2003-2019

0.2.4.4 Parameterübergabe: Call by Reference mittels Referenzen(C++)

Beispiel C++

```
int verdoppeln(int& x) // x ist anderer Name fuer
{                      // Speicherstelle einer
    x = x * 2;         // int Parametervariable
    return x;
}
```

int& spezifiziert:

- Referenz(C++) wird übergeben
- ein „anderer Name“ für außerhalb existierende Variable

```
b = verdoppeln (a);
cout << a << ", " << b;
```

- *Referenz*(C++) von a an Funktion übergeben
- Innerhalb `verdoppeln` ist x ein anderer Name für a
- Rückgabewert der Funktion in b gespeichert
- a *wird verändert*
- Ausgabe: 6, 6

0.2.4.5 Parameterübergabe: Call by Reference mittels Zeigern

Beispiel C / C++

```
int verdoppeln(int *x)//Zeiger auf int
{//Inhalt von x mit 2 multiplizieren
  *x = *x * 2;
  return *x; //Rueckgabe: Inhalt von x
}              //(Typ int)
```

- **int** *x: „x ist *Zeiger* auf **int**“ (lies v. rechts n. links!)
- *x: „Inhalt von x“

© R.-R. Grigat 2003-2019

```
6 int a = 3;
7 int b = verdoppeln (&a);
8 cout << a << ", " << b;
```

- &a: „*Adresse* von a“
- Ausgabe: 6, 6

Name	Adresse	Typ
a	0x34f2	int
b	0x34f6	int
x	0x5284	int *

Tabelle 2: Symboltabelle des Compilers

Adresse	Inhalt
...	...
0x34f2	3
0x34f6	???
...	...
0x5284	0x34f2

Tabelle 3: Daten in `verdoppeln`

0.2.4.6 Probleme von Zeigern

Probleme von Zeigern:

- schlechte Lesbarkeit, z. B. `*x = *x *2;`
- Unterscheidung notwendig, ob Wert oder Adresse übergeben wird, z. B. `f(a); f(*a); f(&a);`
- fehleranfällig, da Gültigkeit von Adressen oft nicht überprüft wird

Daher wird die Verwendung von Referenzen(C++) empfohlen

Bindung des '*' und äquivalente Schreibweisen

- `int* a; int * b; int *c; //a,b,c Zeiger auf int`
- `int* a, b; //b ist kein Zeiger sondern int`

© R.-R. Grigat 2003-2019

0.2.4.7 Konstante Referenz(C++) und Zeiger auf Konstante

Fragestellung:

- *Call by Reference* verwenden zur Parameterübergabe, da schneller als *Call by Value*,
- aber Veränderung der Parameteroriginale verhindern

Lösung in C++:
const verwenden

Beispiel

```
1 int verdoppeln(const int& x)
2 { //x ist Referenz, Wert konstant
3     x = x * 2; //unzulässig!
4     return x;
5 }
```

```
1 int verdoppeln(const int* x)
2 { //x ist Zeiger, Inhalt konstant
3     *x = *x * 2; //unzulässig!
4     return *x;
5 }
```

Compilerfehler, da `x` bzw. `*x` verändert wird, obwohl als ***const*** deklariert.

0.2.4.8 Zeiger auf konstanten Inhalt vs. konstanter Zeiger

Zeiger auf konstanten Speicherinhalt:

- Der Zielinhalt des Zeigers *a* darf nicht per Zuweisung an $*a$ verändert werden
- Nach Änderung von *a* gilt dies nun für neuen Zielinhalt
- Beispiel (beide Schreibweisen sind äquivalent):

```
1 int const* a; // a ist Zeiger auf konstanten int, *a stets const
2 const int* b; // b ist Zeiger auf konstanten int, *b stets const
```

© R.-R. Grigat 2003-2019

Konstanter Zeiger

- Die Adresse des Ziels, auf das der Zeiger zeigt, darf nicht verändert werden.

```
1 int       * const c; // c ist konstanter Zeiger auf variablen  int
2 int const * const c; // c ist konstanter Zeiger auf konstanten int
```

0.2.4.9 Überladen von Funktionen

Problem:

- Funktionsname soll nur Funktionalität beschreiben
- Name soll für verschiedene Datentypen verwendbar sein
- Redundanz in Funktionsnamen vermeiden

```
1 int verdoppeln_int(int x);
2 double verdoppeln_float(float x);
```

Lösung: Überladen von Funktionen

```
1 int verdoppeln(int x);
2 double verdoppeln(float x);
```

Compiler entscheidet mittels *Signatur* (Liste übergebener Parametertypen) einer Funktion, welche Funktion gemeint ist.

Beispiel

```
1 int a = 5;
2 double b = verdoppeln(a);//verdoppeln(int), Ergebnis nach double konvertieren
```

0.2.4.10 Signatur und Typ von Funktionen

Signatur versus (Return-)Typ einer Funktion

- Signatur normaler Funktionen besteht aus Anzahl und Typ der (Aufruf-)Parameter
- Signatur von Template-Funktionen bezieht auch Template-Parameter ein
- Fehlermeldung bei Mehrdeutigkeiten impliziter Typumwandlungen in der Signatur überladener Funktionen

© R.-R. Grigat 2003-2019

- Der (Return-)Typ einer normalen Funktion gehört nicht zur Signatur, denn er ist nicht dem Aufruf zu entnehmen und eine Unterscheidung überladener Funktionen nur anhand ihres Typs ist daher nicht möglich.

```
1 int search(string key);      // Signatur: string, Typ: int
2 string search(string name); // nicht zulässig, Signatur identisch
```

0.2.4.11 Platzhalter auto für Return-Typen von Funktionen

- Platzhalter für Return-Typ, nützlich bei komplexen Typen und Definition von Funktionstemplates
- Return-Typ einer Funktion kann dem Ende des Funktionskopfes nachgestellt explizit angegeben werden.
- Der nachgestellte Return-Typ kann entfallen, wenn Compiler diesen aus return-Anweisung ableiten kann.
  - Alle return-Anweisungen der Funktion müssen dazu exakt gleichen Typ haben.
  - Compiler muss Definition der Funktion vor erstem Aufruf der Funktion kennen.
  - Einem rekursiven Aufruf muss mindestens eine return-Anweisung vorausgehen.

```
1 auto add(float, float) -> double {/* Funktionsblock*/}
```

0.2.5 Modularisierung

Problem:

- bei großen Programmen lange Kompilierzeit
- eine große Quelltextdatei wäre sehr unübersichtlich

Lösung:

- Aufteilung in mehrere Quelltextdateien
- separates Kompilieren jeder Datei
- Dateien nur kompilieren, falls geändert

© R.-R. Grigat 2003-2019

0.2.5.1 Modularisierung in C++

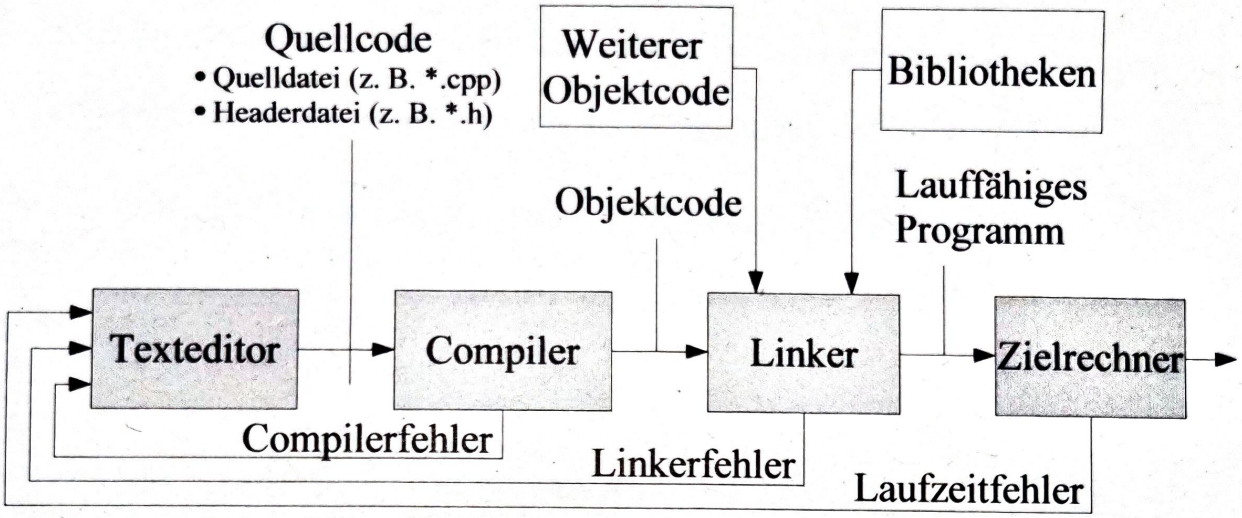


Abbildung 1: C++: Schritte vom Quelltext zum Programm

1. separates Kompilieren jeder Quelldatei in Objekt-Datei
2. Objekt-Dateien können in statischen Bibliotheken zusammengefasst werden
3. *Linker* verbindet Objekt-Dateien und statische Bibliotheken zu ausführbarem Programm

0.2.5.2 Modularisierung in C++

Problem:

- Variablen aus fremden Quelldateien (z. B. `*.cpp`) und Bibliotheken müssen ohne deren erneute Kompilation dem Compiler bekannt gemacht werden

Lösung:

- Deklarationen in *Header-Dateien* (z. B. `*.hpp`) auslagern

© R.-R. Grigat 2003-2019

0.2.5.3 Präprozessor

- Präprozessor (Teil des Cpp-Compilers) führt einfache Textersetzungen durch vor eigentlicher Kompilierung
- Präprozessoranweisungen werden mit # eingeleitet

0.2.5.4 Beispiel zu Header-Dateien in C++

Header-Datei

```
1 //verdoppeln.hpp
2 int verdoppeln(int x);
3 float verdoppeln(float x);
```

cpp-Datei

```
1 //main.cpp
2 #include "verdoppeln.hpp"
3 #include <iostream>
4
5 using namespace std;
6
7 int main()
8 {
9         int a = 2;
10        cout << verdoppeln (a);
11        return 0;
12 }
```

- **#include**"verdoppeln.hpp" fügt Datei verdoppeln.hpp in Quelltext ein
- Unterschied zwischen **#include** "" und **#include** < > :
  - **#include** "" sucht nach Datei im aktuellen Verzeichnis
  - **#include** < > sucht in allen bekannten Verzeichnissen (Suchpfad einstellbar)

© R.-R. Grigat 2003-2019

0.2.5.5 Beispiel zu bedingter Kompilierung in C++

Problem:

- Definitionen und Deklarationen aus einer Header-Datei werden von mehreren Quelldateien (z. B. `*.cpp`) benötigt.
- Definitionen dürfen aber nur einmal kompiliert werden
- `\#pragma once` nicht standardkonform, nicht empfohlen

Lösung:

- *Include-Wächter:* Bedingte Kompilierung mit (Präprozessor-)*Makros*
- Makro-Namen sind nicht gegen Namenskollision geschützt, (Tip: lange und spezifische Namen verwenden)

© R.-R. Grigat 2003-2019

0.2.5.6 Beispiel zu bedingter Kompilierung in C++

Beispiel

```
1 //konstanten.hpp
2 #ifndef KONSTANTEN_H
3 #define KONSTANTEN_H
4 const double PI = 3.14;
5  //Definition von PI
6  //global wirksam
7 #endif // KONSTANTEN_H
```

```
1 //konstanten.cpp
2 #include "konstanten.hpp"
3 /* ... */
```

```
1 //main.cpp
2 #include "konstanten.hpp"
3 /* ... */
```

1. zunächst KONSTANTEN_H nicht definiert, also: **#ifndef** KONSTANTEN_H ist-wahr (**#ifndef** für „if not defined“)
2. definiere Makro KONSTANTEN_H
3. führe alles aus bis **#endif**
4. Header-Datei wird erneut eingebunden
5. KONSTANTEN_H ist schon definiert, also: **#ifndef** KONSTANTEN_H ist falsch
6. überspringe alles bis **#endif**, also keine Reinitialisierung der Konstanten PI

0.2.5.7 Deklaration versus Definition von Grundtypen

extern vor Deklaration von Grundtypen

- Übersetzungseinheit: Quelldatei einschließlich includes
  - *Interne Bindung*: sichtbar nur in lokaler Übersetzungseinh.
  - *Externe Bindung*: sichtbar in allen Übersetzungseinheiten

© R.-R. Grigat 2003-2019

- **const**, constexpr, **inline** besitzen *interne* Bindung, **extern** schaltet externe Bindung ein
- Grundtypen besitzen *externe* Bindung, **static** schaltet auf interne Bindung um
- Unbenannter Namespace: entspricht interner Bindung

```
1 int a;           // Definition, bedeutet "int a=0;", externe Bindung
2 int a = 1;       // Fehler, Redefinition
3 extern int b;    // nur Deklaration, keine Definition
4 extern float b; // Fehler, falscher Typ
5 extern int b;    // ok, Redeklaration
6 b = 5;           // Fehler, b noch nicht definiert
7 const int c = 1;       // Konstanten nicht extern sichtbar, interne Bindung
8 extern const int d=1; // externe Bindung der Konstante
9 extern int e = 1;      // Definition, "extern" ist unwirksam
```

0.2.5.8 Speicherklasse von Objekten

Speicherklassen

- programmglobal (global), modulglobal oder lokal
- festgelegt durch
  - *Position* der Deklaration innerhalb Quelldatei
  - Speicherklassen-*Spezifizierer* (**extern**, **static**)
  - **const**, constexpr, Typalias, **static** erzeugen *interne* Bindung, sollten in Header-Dateien stehen zur Konsistenz zwischen Übersetzungseinheiten.

Merkmale von Speicher

- Lebensdauer (storage duration)
- Geltungsbereich (scope)

Beispiel: KP11_1_externSpeicherklassen-Spezifizierer, KP11_2_staticVariables, KP11_3_auto_register

© R.-R. Grigat 2003-2019

0.2.5.9 Programmglobal versus modulglobal

Programmglobal (global) und modulglobal

- Deklaration außerhalb jeder Funktion
- Lebensdauer gesamte Programmausführung (statisch, permanent)

Programmglobal (global)

- Geltungsbereich gesamtes Programm
- dem Linker bekannt
- unterteilbar durch Namensbereiche
- *extern* vor jeder Deklaration einfügen (nicht vor Definition)

Modulglobal

- Geltungsbereich nur Modul, dem Linker nicht bekannt (cpp-Quelldatei samt #include, Übersetzungseinheit)
- *static* vor Definition (nicht Deklaration) einfügen

0.2.5.10 Lokaler Speicher

Nicht-statischer (automatischer) lokaler Speicher

- Definition innerhalb eines Geltungsbereichs {}, z.B. Block oder Funktion
- Lebensdauer endet bei Verlassen des Geltungsbereichs
- Automatischer Speicher stammt vom Stack (Stapel)

Statischer (permanenter) lokaler Speicher

- Definition innerhalb eines Geltungsbereichs {}, z.B. Block oder Funktion
- *static* vor der Definition einfügen
- Lebensdauer ist statisch (permanent)
- Sichtbarkeit endet bei Verlassen des Geltungsbereichs
- Erneut sichtbar bei Wiederbetreten des Geltungsbereichs

© R.-R. Grigat 2003-2019

0.2.5.11 Externe und statische Funktionen

Externe Funktionen

- Programmglobale Funktion
- Kein Spezifizierer *static* vor Definition
- Speicherklassen-Spezifizierer *extern* vor Deklarationen

Statische Funktionen

- Modulglobale Funktion
- Speicherklassen-Spezifizierer *static* vor Definition (und ggf. Deklaration)
- Deklaration nötig bei Aufruf vor der Definition im Modul
- Lokale Funktionen unzulässig (keine Funktionsdefinition innerhalb einer anderen Funktion)

Beispiel: Geltungsbereich

0.2.5.12 Namensräume (Namespaces)

Problem:

- verschiedene Bibliotheken könnten denselben Namen für Funktionen oder Variablen verwenden

```
1 //tuvision.hpp
2 const int ANZAHL = 5;
3 void ausgabe(int x);
```

```
1 //infing2.hpp
2 const int ANZAHL = 7;
3 void ausgabe(int x);
```

```
1 //main.cpp
2 #include "tuvision.hpp"
3 #include "infing2.hpp"
4 //Fehlermeldung wegen
5 //Mehrdeutigkeit von ANZAHL
```

© R.-R. Grigat 2003-2019

```
6 int main()
7 {
8   ausgabe(ANZAHL);
9   return 0;
10 }
```

Lösung in C++:
getrennte Namensräume

Beispiel

```
1 //tuvision.hpp
2 namespace tuvision
3 {
4   const int ANZAHL = 5;
5           //tuvision::ANZAHL
6   void ausgabe(int x);
7           //tuvision::ausgabe
8 }
```

```
1 //infing2.hpp
2 namespace infing2
3 {
4   const int ANZAHL = 7;
5           //infing2::ANZAHL
6   void ausgabe(int x);
7           //infing2::ausgabe
8 }
```

Auflösung des Namensraums

- scope-Operator ::
- **using** Deklaration oder **using namespace** Direktive

```
1 tuvision::ausgabe(infing2::ANZAHL);
```

```
1 using tuvision::ausgabe;
2 // Deklaration: Funktion ausgabe
3 // aus tuvision ist Default
4 ausgabe(infing2::ANZAHL);
```

© R.-R. Grigat 2003-2019

```
using namespace tuvision;
// Direktive: alle Definitionen aus
// tuvision sind Default
ausgabe(ANZAHL);
```

Beispiel: KP11_4_namespaceDemo, KP11_5_using_namespace

0.2.5.13 Unbenannter Namensraum

Unbenannter Namensraum

- Kapselung von Übersetzungseinheiten hinsichtlich globaler Variablen durch unbenannte Namensräume.

```
namespace {              // anonymer Namensraum
int a;
void f() { /* ... */ }
int g() { /* ... */ }
}
```

- Unbenannte Namensräume verhalten sich, als hätten sie einen nur in der Übersetzungseinheit bekannten eindeutigen Namen, hier $$$, mit **using**-Direktive.

```
namespace $$$ {          // $$$ nur lokal bekannt
    int a;
    void f() { /* ... */ }
    int g() { /* ... */ }
}
using namespace $$$;
```

0.2.5.14 Verschachtelte Namensräume

```
void h();                // h()
namespace X {
    void g();            // X::g()          // äußerer Namensraum
    namespace Y {
        void f();        // X::Y::f()       // innerer Namensraum
        void ff();       // X::Y::ff()
    } // X::Y
} // X
```

© R.-R. Grigat 2003-2019

```
1  void X::Y::ff() {
2  f(); g(); h();      // OK, Zugriff auf Umgebung
3  }
4  void X::g() {
5  f();                // Fehler: kein f() in X
6  Y::f();             // OK
7  }
8  void h() {
9  f();                // Fehler: keine globale f()
10 Y::f();             // Fehler: kein globaler Y
11 X::f();             // Fehler: kein f() in X
12 X::Y::f();          // OK
13 }
```

0.3 Klassen und Objekte

0.3.1 Schnittstellen

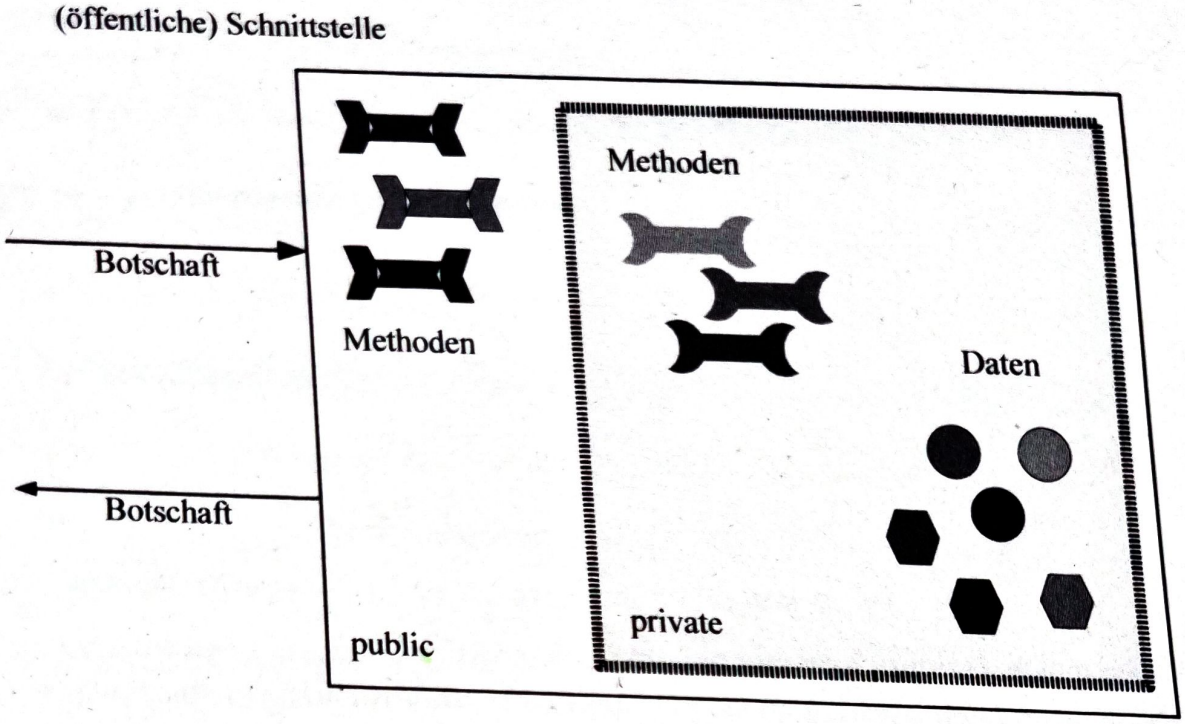


Abbildung 2: Ein Objekt

- Programm besteht aus Objekten
- Objekt besteht aus
  - Daten
  - Methoden, die diese Daten verändern können
- Kommunikation zwischen Objekten mittels Methoden
- Objekte sind Instanzen von *Klassen*
- *Kapselung*
  - Trennung von Schnittstelle und Inhalt
  - Zugriff auf Daten über genau definierte Schnittstellen
  - Grundprinzip objektorientierter Programmierung

© R.-R. Grigat 2003-2019

- *Objekte*
  - sind individuelle Instanzen ihrer Klassen
  - Jedes Objekt besitzt eigenen Speicherbereich für Daten
  - Methoden sind immer einem Objekt zugeordnet

0.3.1.1 Abstrakter Datentyp (ADT)

Merkmale einer Datenstruktur (Datentyp)
Eine Datenstruktur bzw. ein Datentyp besteht aus

- Menge (von Daten)
- Operationen

Merkmale eines abstrakten Datentyps

- Definition: Abstrakter Datentyp spezifiziert Schnittstelle zu einer Datenstruktur anhand mathematischer Beschreibung.
- Beschreibung des Datentyps mit (Datentyp-) *Axiomen*
  - beschreiben nur Verhalten das Datentyps
  - keine Aussage über konkrete Implementierung
  - nicht verwechseln mit Axiomen der Prädikatenlogik

0.3.1.2 Beispiel: Abstrakter Datentyp (ADT) Stack oder Stapel

Beispiel: ADT Stack (oder Stapel)

$$\begin{aligned} \text{push:}\quad & S \times D \to S \\ & push\,((d_1,\ldots,d_n),d) = (d_1,\ldots,d_n,d) \\ \text{pop:}\quad & S \to S \times D \\ & pop\,(push\,(s,d)) = (s,d) \\ \text{top:}\quad & S \to D \\ & top\,(d_1,\ldots,d_n,d) = d \end{aligned}$$

- Keine Aussage, was ein Stack ist
- Kein Hinweis auf Implementierung

© R.-R. Grigat 2003-2019

0.3.2 Klassen

0.3.2.1 Klassen in C++

Klassendefinition

```
1 class CAuto
2 {
3   public:  // Modifikator "fuer alle Funktionen sichtbar"
4     std::string getFarbe(); //Methode (in C++: "Memberfunktion")
5     void lackiere(std::string farbe); // Memberfunktion
6   private: // Modifikator "nur für Memberfunktionen dieser Klasse sichtbar"
7           std::string m_farbe; // Daten (in C++: "Membervariablen")
8 };
```

- Modifikator **public:** Öffentliche Schnittstelle, Methoden und Daten überall sichtbar (z.B. Aufruf aus main-Funktion)
  - `getFarbe()` soll Inhalt von `m_farbe` liefern
  - `lackiere(std::string farbe)` soll Inhalt von `m_farbe` ändern
- Modifikator **private:** Methoden und Daten unsichtbar[1] außerhalb der Klasse (z.B. *kein* Aufruf aus main-Funktion)

0.3.2.2 Klassen und Struct in C++

class** versus **struct

```
1 class CAuto
2 {
3     int a;     // Default: private
4   private:
5     int m_b;   // private
6   public:
7     int c;     // public
8 };
```

[1] Default-Eigenschaft von Methoden und Daten ist **private:**

© R.-R. Grigat 2003-2019

```
1 struct CAuto
2 {
3     int a;      // Default: public
4   private:
5     int m_b;    // private
6   public:
7     int c;      // public
8 };
```

- In **struct** sind Member per Default **public**
- In **class** sind Member per Default **private**
- Ansonsten sind **struct** und **class** *identisch*

Beispiel: KP13_1_classKonto, KP13_3_structVertreter, KP13_4_unionWordByte
Mehr zu Zugriffsrechten in Abschnitt 0.3.5.6

0.3.2.3 UML-Repräsentation von Klassen

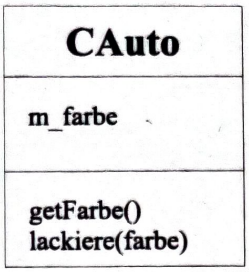


Abbildung 3: UML-Diagramm: Klasse

Erläuterung

- oben: Klassenname
- mittig: Daten (werden oft weggelassen, in C++ auch „Membervariablen“ genannt)
- unten: Methoden (in C++: „Memberfunktion“ oder „Elementfunktion“)
- falls nur Klassenname interessiert; Methoden und Daten weglassen

UML: Unified Modeling Language

0.3.2.4 Beispiel: Komplexe Zahlen in C++

Klassendefinition

```
1  //ccomplex.hpp
2  class CComplex
3  {
4  public:
5    double re();
6    double im();
7    void add(const CComplex& z);
8    void print();
9  private:
10   double m_re;
11   double m_im;
12 };
```

© R.-R. Grigat 2003-2019

Klassen*definition* enthält (nur) *Deklaration* der Memberfunktionen

Beispiel: CComplex

Definition der Memberfunktionen

```
1  //ccomplex.cpp
2  double CComplex::re()
3  {
4    return m_re;
5  }
6  double CComplex::im()
7  {
8    return m_im;
9  }
10 void CComplex::add(const CComplex& z)
11 {
12   m_re += z.re();
13   m_im += z.im();
14 }
15 void CComplex::print()
16 {
17   cout << m_re <<"+"<< m_im <<"j"<< endl;
18 }
```

© R.-R. Grigat 2003-2019

0.3.2.5 Definition der Memberfunktion

```
1 double CComplex::re()
2 {
3   return m_re;
4 }
```

- Funktionsdefinition besteht aus
  - Rückgabetyp **double**
  - Funktionsname `CComplex::re`
  - Parameterliste `()`
  - Anweisungsblock `{ ... }`
- Scope-Operator `::`
  - zeigt an, zu welcher Klasse Memberfunktion oder -variable gehört
  - hier: Memberfunktion `re()` aus Klasse `CComplex`

0.3.2.6 Zugriff auf Membervariablen / Memberfunktionen

Zugriff auf Memberfunktion

```
1 double CComplex::re()
2 {
3   return m_re;
4 }
```

- `z` sei Objekt der Klasse `CComplex`
- Zugriff auf Elemente von `z` mit Punktoperator `.`
  - Beispiel: `z.re()` ruft Memberfunktion `re()` aus Objekt `z` auf
- Innerhalb eines Objekts direkter Zugriff auf Membervariablen / Memberfunktionen möglich (z. B. `m_re`)

© R.-R. Grigat 2003-2019

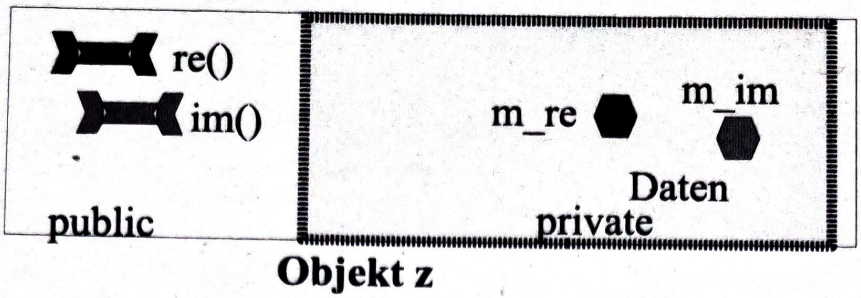


Abbildung 4: Das Objekt z

0.3.2.7 Konstante Memberfunktion

Problem:
Verhindern, dass in einer Memberfunktion der Zustand des eigenen Objekts verändert wird

Lösung:
Konstante Memberfunktion verwenden

Beispiel:

```
1 double CComplex::re() const; // re() darf eigenes Objekt nicht veraendern
```

- `const` zeigt an, dass Membervariablen des eigenen Objekts in dieser Funktion nicht verändert werden dürfen
- `const` muss sowohl in Funktionsdeklaration als auch in Funktionsdefinition angegeben werden,
- nicht verwechseln mit konstanten Parametern (s. u.)

0.3.2.8 Konstante Parameter vs. konstante Memberfunktion

Beispiel

```
1 //cauto.hpp
2 class CAuto
3 {
4   public:
5     std::string getFarbe() const; // konstante Memberfunktion
```

© R.-R. Grigat 2003-2019

```
6     void lackiere(const std::string farbe); //konstanter Parameter
7   private:
8     std::string m_farbe;
9 };
```

- `void lackiere(const std::string farbe)` besitzt *konstanten Parameter*
  - `farbe` ist konstanter Parameter
  - Membervariable `m_farbe` darf verändert werden
- `getFarbe()` ist *konstante Memberfunktion*
  - Membervariable `m_farbe` darf *nicht* verändert werden
- Konstante Memberfunktion kann auch konstante Parameter besitzen

0.3.2.9 Selbst-Referenz mit Zeiger `this`

Problem:

- Memberfunktion `z.add(z1)` soll Referenz(C++) auf eigenes Objekt z zurückliefern
- ermöglicht `z.add(z1).add(z2)`
  - erst z1 hinzuaddieren, zum Ergebnis z2 hinzuaddieren

Lösung:

- Jedes Objekt enthält einen Zeiger auf sich selbst,
- dieser Zeiger ist in Membervariable **`this`** abgelegt.

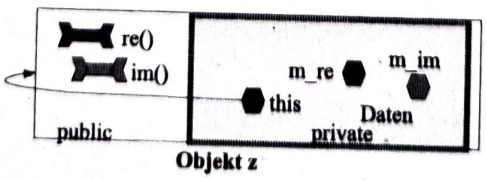


Abbildung 5: Das Objekt z mit **`this`**-Zeiger

© R.-R. Grigat 2003-2019

0.3.2.10 Beispiel zu this

Beispiel

```
1 CComplex& CComplex::add(const CComplex& zn)
2 {
3   m_re += zn.re();
4   m_im += zn.im();
5   return *this;
6 }
```

- `this` ist Zeiger auf eigenes Objekt
- `*this` ist Inhalt des eigenen Objekts
- Das Objekt wird als Referenz(C++) `CComplex&` zurückgegeben
- Damit ist `z.add(z1).add(z2)` möglich
  - Zunächst wird die Memberfunktion `add(z1)` von `z` aufgerufen
  - Von dem Rückgabeobjekt (hier Referenz auf `z`, also nur anderer Name für `z`) wird `add(z2)` aufgerufen

0.3.2.11 Der Konstruktor

Der Konstruktor

- Konstruktor wird zur Erzeugung eines Objektes ausgeführt
- *Initialisierung* von Membervariablen im Konstruktor oder (seit C++11) durch *direkte Initialisierung* in Klassendefinition, Konstruktor hat ggf. Vorrang
- Keine automatische Initialisierung von Membervariablen
  - Variablen von Compiler *nicht* automatisch initialisiert
  - Compiler erzeugt auch keine Warn- oder Fehlermeldung
  - Inhalt ohne Initialisierung zufällig!
- Konstruktor ist eine Funktion ohne Rückgabewert
- Konstruktorname identisch zu Klassenname

© R.-R. Grigat 2003-2019

- Compiler stellt Default-Konstruktor ohne Parameter zur Verfügung, solange kein eigener Konstruktor definiert wird
- *Delegation* von Konstruktoren (C++11), Konstruktor darf anderen Konstruktor der Klasse aufrufen

0.3.2.12 Beispiel Konstruktor C++

Beispiel Konstruktor C++

```
1 //ccomplex.hpp
2 class CComplex
3 {
4 public:
5   CComplex(double r, double i); // Deklaration des Konstruktors
6 /* ... */
7 };
```

```
1 //ccomplex.cpp
2 CComplex::CComplex(double r, double i)
3 {                                  // Definition des Konstruktors
4   m_re=r;
5   m_im=i;
6 }
```

```
1 //main.cpp
2   CComplex mycomplex(3.0,4.0);
```

Beispiel: KP14_1_konstruktor, KP14_3_defaultMethoden

0.3.2.13 Default-Argumente in Funktionen (einschließlich Konstruktoren)

Default-Argument

- Es können in C++ Default-Argumente für Parameter von Funktionen spezifiziert werden (also auch für Konstruktoren)
- Default-Werte nur für hinten stehende Argumente, Ersetzung von rechts nach links
- alternativ: Überladen des Konstruktors

© R.-R. Grigat 2003-2019

Beispiel

```
//ccomplex.hpp
CComplex::CComplex(double r, double i=0); // Konstruktor-Deklaration
```

```
//main.cpp
  CComplex mycomplex(3.0); // Initialisierung i = 0
```

0.3.2.14 =Delete und =Default

=Delete und =Default

- Delete: Definition einer Memberfunktion sperren
- Beispiel: Kopierkonstruktor und Zuweisung sperren

```
//ccomplex.hpp
CComplex (const CComplex&) = delete; // Kopierkonstruktor sperren
operator=(const CComplex&) = delete; // Kopierzuweisung sperren
CComplex(CComplex&&) = delete; // Movekonstruktor sperren
CComplex& operator=(CComplex&&) = delete; // Movezuweisung sperren
```

- Default-Variante zurückbekommen, selbst wenn eigene Konstruktoren definiert wurden

```
CComplex (const CComplex&) = default; // Kopierkonstruktor aktivieren
operator=(const CComplex&) = default; // Kopierzuweisung aktivieren
CComplex(CComplex&&) = default; // Movekonstruktor aktivieren
CComplex& operator=(CComplex&&) = default; // Movezuweisung aktivieren
```

Beispiel: KP14_4_defaultDelete

0.3.2.15 Der Destruktor

- wird bei Zerstörung eines Objektes aufgerufen
- ermöglicht Aufgaben wie
  - Freigabe nicht mehr benötigten dynamischen Speichers
  - Schließen offener Dateien

© R.-R. Grigat 2003-2019

- Destruktor ist eine Funktion ohne Rückgabewert
- Destruktorname identisch zu ~Klassenname
- immer ohne Parameter
- Compiler stellt Default-Destruktor zur Verfügung, falls kein eigener Destruktor definiert

Beispiel C++

```
// ccomplex.hpp
class CComplex
{
public:
  ~CComplex(); // Deklaration eines Destruktors
  /* ... */
};
```

```
// ccomplex.cpp
CComplex::~CComplex() // Definition eines Destruktors
{} // hier ohne Funktionalitaet
   // default-Destruktor auch moeglich
```

Beispiel: KP14_2_konstruktorDestruktor

0.3.3 Objekte

0.3.3.1 Speicherplatz in C++

In C++ Unterscheidung zwischen

- statischem Speicher
  - vom Linker für die gesamte Dauer der Programmausführung angelegter Speicherplatz
- automatischem Speicher („Stack“)
  - Speicher für Variable wird bei Definition „automatisch“ bereitgestellt
  - Speicher wird nach Verlassen ihres Gültigkeitsbereichs {} freigegeben
- dynamischem Speicher („Heap“)

© R.-R. Grigat 2003-2019

- Speicher für Variable wird durch Programm explizit angefordert (mit **new**)
- Speicher bleibt auch nach Verlassen eines Gültigkeitsbereichs {} belegt
- Speicher muss explizit mit **delete** freigegeben werden

0.3.3.2 Objekte im automatischen Speicher

Beispiel

```
1 { // lokales Objekt
2   CComplex mycomplex(3.0,4.0);
3 /* ... */
4 } // mycomplex wird hier wieder zerstoert
```

- Objekt wird zerstört, sobald Gültigkeitsbereich verlassen wird
- Gültigkeitsbereich: Innerhalb { }
- Beim Erreichen von } wird Destruktor aufgerufen
- Objekt danach nicht mehr existent

0.3.3.3 Objekte ohne Parameter

Beispiel

```
1   CAuto erstwagen; // Aufruf des parameterlosen Konstruktors
2                    // zur Erzeugung des Objekts erstwagen
```

Achtung:

- *nicht* `CAuto erstwagen();`
- dies wäre Funktionsdeklaration!

© R.-R. Grigat 2003-2019

0.3.3.4 Objekte im dynamischen Speicher in C++

Beispiel

```
1 CComplex *z = new CComplex(1.0, 2.0);
2 (*z).print();
3 delete z;
```

- Objekt wird mit Operator **new** erzeugt
- **new** liefert Zeiger auf (neu erstelltes) Objekt im dynamischen Speicher (Heap)
- Objekt existiert über Gültigkeitsbereich hinaus
- Objekt muss mit Operator **delete** explizit gelöscht werden
- Vergessen von **delete**: Ursache von „Speicherlecks“ (Freier Arbeitsspeicher wird immer kleiner)

Problem: Lesbarkeit

```
1 (*z).print();
```

Abhilfe: alternative Schreibweise

```
1 z->print();
```

- Beide Schreibweisen sind äquivalent
- Gebräuchlich ist ausschließlich ->

Beispiel: KP13_2_pointerToObject, KP21_1_dynamisch_Standardtypen, KP21_2_dynamisch_Klassen, KP21_3_dynamisch_Arrays, KP21_4_einfachVerketteteListe

0.3.3.5 Statische Variablen und Funktionen

Statische Variable Definition außerhalb aller Funktionen und Objekte. Sichtbarkeit innerhalb des Moduls (modulglobal), also innerhalb der Sourcecode-Datei mit der Definition der Variablen.

Statische Membervariable Speicherbereich ist einer Klasse (Bauplan) zugeordnet, nicht einem Objekt (Instanz). Zugriff auf private statische Membervariablen möglich schon vor Existenz eines Objekts durch eine statische Memberfunktion dieser Klasse.

© R.-R. Grigat 2003-2019

Statische Memberfunktion ist einer Klasse zugeordnet, nicht einem Objekt. Aufruf möglich vor Existenz eines Objekts, dient insbesondere Zugriff auf statische Membervariablen dieser Klasse.

0.3.3.6 Statische Membervariablen

- Jede Instanz einer Klasse erhält eigenen Speicherbereich
- Ausnahme: *Statische Membervariablen* belegen nur einmal Speicherplatz
  - Alle Instanzen greifen auf dieselbe Membervariable zu
  - Statische Membervariablen werden im statischen Speicher abgelegt (vgl. 0.3.3.1)

Anwendungsbereich statischer Membervariablen:

- Synchronisation von Objekten einer Klasse für gemeinsame Ressource
- Gemeinsame Einstellungen (z. B. Ausgabeformat) für alle Objekte einer Klasse

0.3.3.7 Statische Membervariablen: Beispiel C++

Beispiel

```
1  //ccomplex.hpp
2  class CComplex
3  {
4  public:
5    CComplex(double r, double i);
6    ~CComplex();
7    double re();
8    double im();
9    void print();
10 private:
11   double m_re;
12   double m_im;
13   static bool s_polar;
14    //statische Membervariable
15 };
```

© R.-R. Grigat 2003-2019

```
1 //ccomplex.cpp
2 bool CComplex::s_polar = false;
3                 //Initialisierung
```

- `CComplex::s_polar` gibt an, ob Ausgabe in Polarkoordinaten erwünscht
- `CComplex::s_polar` existiert genau einmal, unabhängig von Anzahl Instanzen der Klasse
- Initialisierung muss genau einmal erfolgen

0.3.3.8 Statische Memberfunktionen

- können aufgerufen werden, auch wenn noch keine Instanz der Klasse existiert
- können sinnvoll *nur* auf statische Membervariablen und auf Variablen ihrer Signatur zugreifen (weil sonst der Name eines Objekts der Klasse direkt angesprochen werden müsste)

Anwendungsbereich:

- Schnittstelle für statische Membervariablen

Beispiel: KP15_3_statischeDatenelemente

0.3.3.9 Statische Memberfunktionen: Beispiel C++

Beispiel

```
1 //ccomplex.hpp
2 class CComplex
3 {
4 public:
5 /* ... */
6   void print();
7   static void setPolar(bool format);
8     //statische Memberfunktion,
9     //Deklaration
```

© R.-R. Grigat 2003-2019

```
10 private:
11 /* ... */
12   static bool s_polar;
13     //statische Membervariable
14 };
```

```
1 //ccomplex.cpp
2 bool CComplex::s_polar = false; //Init.
3 void CComplex::setPolar(bool format)
4 { //stat. Memberfunktion, Definition
5   s_polar=format;
6 }
```

```
1 // main.cpp
2 int main()
3 { // Aufruf statischer Memberfunktion
4   CComplex::setPolar(true);
5   CComplex *z = new CComplex(1.0, 2.0);
6   z->print();
7   delete z;
8   return EXIT_SUCCESS;
9 }
```

- `CComplex::setPolar` öffentliche Schnittstelle für Zugriff auf `s_polar`
- Funktion kann aufgerufen werden, auch wenn kein Objekt der Klasse existiert

0.3.4 Komposition

Definition: Komposition

- Objekt enthält weitere Objekte als Klassenelement („Memberobjekt“)
- „hat-ein“ Beziehung

Abbildung 6: UML-Diagramm: Komposition: „Auto hat Hupe“

© R.-R. Grigat 2003-2019

0.3.4.1 Komposition in C++

Beispiel C++: Auto hat Hupe

```
1  //chupe.hpp
2  class CHupe
3  {
4    public:
5      CHupe(std::string ton,
6          int staerke);
7      ~CHupe();
8    private:
9      std::string m_ton;
10     int m_staerke;
11 };
```

```
1  //cauto.hpp
2  class CAuto
3  {
4    public:
5      CAuto();
6      std::string farbe();
7      void lackiere(std::string farbe);
8      CHupe m_hupe; // Auto hat Hupe
9    private:
10     std::string m_farbe;
11 };
```

Beispiel: KP15_1_Komposition, KP15_2_KonstanteTeilobjekte

0.3.4.2 Initialisierung von Memberobjekten

Problem bei Komposition

- Initialisierung von Memberobjekten mit Parametern?

Lösung

- Initialisierung des Memberobjekts CHupe im Konstruktor von CAuto
- mit *Initialisierungsliste* hinter Doppelpunkt :

© R.-R. Grigat 2003-2019

Beispiel C++

```
1 //cauto.cpp
2 CAuto::CAuto() :              //Beginn Initialisierungsliste für CAuto
3   m_hupe("hohes C", 128),
4   m_farbe("Schwarz")
5 {
6     cout << "Auto initialisiert" << endl;
7 }
```

0.3.4.3 Initialisierungsliste

- ermöglicht Initialisierung von Objekten
- auch für *Initialisierung von Variablen* verwenden!
  - **const** Elemente müssen hier initialisiert werden
- Liste wird mit : eingeleitet
- Listenelemente werden mit , getrennt

Beispiel: Komplexe Zahlen in C++

```
1 //ccomplex.hpp
2 class CComplex
3 {
4 public:
5   CComplex(double r, double i);
6 private:
7   double m_re;
8   double m_im;
9   const double PI;
10 };  // konstantes Klassenelement
```

```
1 //ccomplex.cpp
2 CComplex::CComplex(double r, double i).
3   :           // Initialisierungsliste
4     m_re(r),
5     m_im(i),
6     PI(3.14)
7 {}
```

0.3.4.4 Objektorientierte Programmierung: Nomenklatur in C++

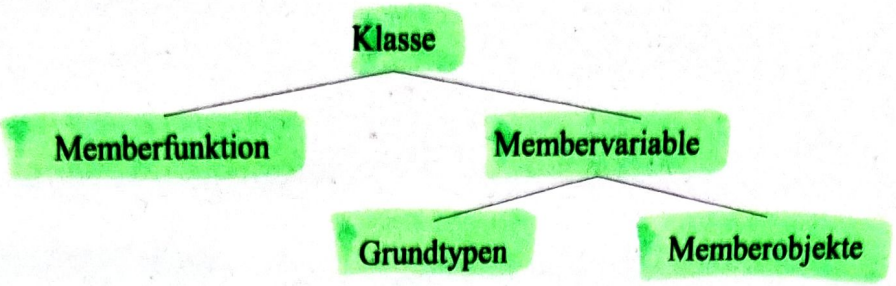


Abbildung 7: Nomenklatur für Klassen in C++

- Klasse in C++ enthält
  - Memberfunktionen (allgemein: Methoden)
  - Membervariablen (allgemein: Daten)
    - Grundtypen, z. B. `bool`, `int`, `double`
    - Memberobjekte (Stichwort: Komposition)
- Für „Member“ wird synonym auch „Element“ verwendet (z. B. Elementfunktion statt Memberfunktion)

0.3.4.5 Statische *Objekt*komponenten als Membervariable

Statische Memberobjekte

- Memberobjekte gehören zu den Membervariablen
- Memberobjekte können *statisch* deklariert werden
- Für statische Memberobjekte gilt das gleiche wie für statische Membervariablen *(s. 0.3.3.6)*
- Statische Funktionen können auch auf statische Memberobjekte zugreifen

0.3.5 Vererbung

Definition: Vererbung (Ableitung)

- Abgeleitete Klasse besitzt zusätzlich (d.h. erbt) die Memberfunktionen und Membervariablen der Basisklasse

© R.-R. Grigat 2003-2019

- „ist-ein“-Beziehung
- Abgeleitete Klasse kann ihrerseits Basisklasse für weitere Vererbung sein
- Vererbung kann beliebig tief geschachtelt werden (Klassenhierarchie)

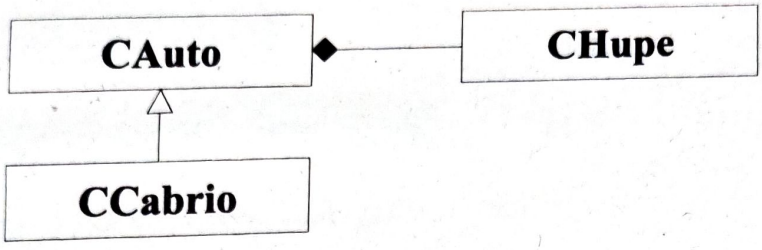


Abbildung 8: UML-Diagramm: Vererbung (und Komposition): „Cabrio ist ein Auto“

0.3.5.1 Vererbung: Beispiel C++

Beispiel C++

```
1 //cauto.hpp
2 class CAuto
3 {
4   public:
5     CAuto();
6     CHupe m_hupe; // Auto hat Hupe
7 };                // (Komposition)
```

```
1 //chupe.hpp
2 class CHupe
3 {
4   public:
5     void hupen(int laenge);
6   private:
7     std::string m_ton;
8     int m_staerke;
9 };
```

```
1 //ccabrio.hpp
2 class CCabrio : public CAuto
3 {                 //Cabrio ist Auto
```

© R.-R. Grigat 2003-2019

```
4   public:          //Vererbung
5     CCabrio();
6     ~CCabrio();
7   };
```

```
1  //main.cpp
2    CCabrio sportwagen;
3    sportwagen.m_hupe.hupen(5);
```

- CCabrio *„ist ein“* CAuto
- Hupe wird vererbt auf CCabrio

Beispiel: KP23_1_vererbung_kfz, KP23_1_vererbung_kfz_Aufgabe

0.3.5.2 Mehrfachvererbung

Definition: Mehrfachvererbung

- Eine Klasse ist aus mehreren Basisklassen abgeleitet
- Erlaubt in C++
- Nicht möglich in Java

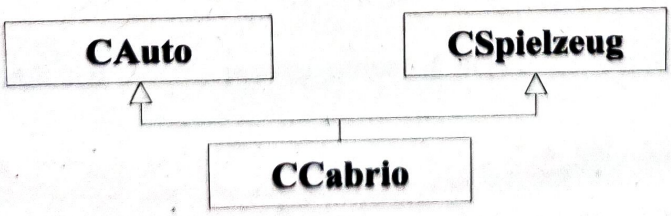


Abbildung 9: UML-Diagramm: Mehrfachvererbung: „Cabrio ist ein Auto und ein Spielzeug“

© R.-R. Grigat 2003-2019

0.3.5.3 Mehrfachvererbung, Initialisierung Basisklasse

Beispiel C++

```
1 //ccabrio.hpp
2 class CCabrio : public CAuto,
3                  public CSpielzeug
4 {                //Mehrfachvererbung
5   public:
6     CCabrio();
7     ~CCabrio();
8 };
```

```
1 //ccabrio.cpp
2 CCabrio::CCabrio()
3   : CAuto(),    //Initialisierung
4     CSpielzeug()//der Basisklassen
5 {
6 /* ... */
7 }
```

Initialisierung der Basisklasse

- Keine direkte Initialisierung von Membern der Basisklassen
- Konstruktor von `CCabrio` ruft über Initialisierungsliste Konstruktoren der Basisklassen `CAuto` und `CSpielzeug` auf

Beispiel: KP27_1_zweiBasisklassen_Wohnwagen

0.3.5.4 Mehrfache indirekte Basisklassen

Mehrfache indirekte Basisklassen

- Indirekte Basisklassen A einer Klasse D existieren mehrfach im Speicher, Gefahr von Mehrdeutigkeiten.

```
1 class B : public A, public A {};  // Fehler
2 class B : public A {};  // nicht-virtuelle Basisklasse A
3 class C : public A {};
4 class D : public B, public C {};  // A ist in D doppelt vorhanden
```

© R.-R. Grigat 2003-2019

- Abhilfe durch *virtuelle* Basisklasse, diese existiert nur einfach im Speicher, Mehrdeutigkeiten sind verhindert.

```
1  class B : public virtual A {};  // virtuelle Basisklasse A
2  class C : public virtual A {};
3  class D : public B, public C {};  // A nur einmal vorhanden
```

- Initialisierung von A durch Konstruktoraufruf in D.

0.3.5.5 Virtuelle Basisklasse

Virtuelle Basisklasse

- Initialisierung virtueller Basisklasse A erfolgt über *tiefste* Ableitungsstufe, hier D. Konstruktoraufrufe von A in B und C bleiben wirkungslos.
- Ruft D keinen Konstruktor von A auf, dann muss A einen Default-Konstruktor besitzen, dieser wird ausgeführt. Konstruktoraufrufe von A in B und C weiterhin wirkungslos.
- Virtuelle Basisklasse A bleibt auch bei weiteren Ableitungen virtuell.
- Indirekte Basisklasse kann *nicht* nachträglich als virtuell deklariert werden, hier also nur in B und C.
- Zuerst werden alle *virtuellen* Basisklassen initialisiert, dann die *nicht-virtuellen* Basisklassen.

Beispiel: KP27_2_virtuelleBasisklasse_kombi

0.3.5.6 Zugriffsrechte auf Memberfunktionen und -variablen

Wiederholung: Modifikatoren für Zugriffsrechte

public: Methoden und Daten der Klasse für alle Funktionen (z.B. `main`-Funktion) sichtbar

private: Methoden und Daten der Klasse nur für Methoden der eigenen Klasse sichtbar (empfohlen für Daten)

protected: Methoden und Daten der Klasse nur für Methoden der eigenen Klasse und der davon abgeleiteten Klassen sichtbar (nur für Methoden sinnvoll)

```
1 class CAuto
2 {
3   public:
4     CAuto();
5     CAuto(std::string farbe);
6   private:
7     std::string m_farbe;
8 };
```

- Methoden aus `CCabrio` können *nicht* auf `m_farbe` zugreifen
- Abhilfe mit **`protected:`** statt **`private:`**

0.3.5.7 Wirkung der Modifikatoren auf Objekte

- Ein Objekt einer *abgeleiteten Klasse* `CCabrio` kann mit ihren Memberfunktionen die **`protected`** Member einer Basisklasse `CAuto`
  - nur in (ggf. anderen) Objekten ihres eigenen Typs `CCabrio` erreichen.
  - *nicht* in Objekten einer Geschwisterklasse (z.B. `CBus`) derselben Basisklasse `CAuto` erreichen.
  - *nicht* in Objekten der Basisklasse `CAuto` erreichen.
- **Ausnahme:** Ein Objekt einer *abgeleiteten Klasse* `CCabrio` kann mit ihren von der Basisklasse `CAuto` *geerbten* Memberfunktionen die **`protected`** *und* **`private`** Member der Basisklasse `CAuto`
  - in (ggf. anderen) Objekten ihres eigenen Typs `CCabrio` erreichen.
  - in Objekten einer Geschwisterklasse (z.B. `CBus`) derselben Basisklasse `CAuto` erreichen und
  - in Objekten der Basisklasse `CAuto` erreichen.

© R.-R. Grigat 2003-2019

0.3.5.8 Wirkung der Modifikatoren auf Objekte (2)

- Sinngemäß kann ein Objekt der *Basisklasse* `CAuto` mit seinen Memberfunktionen die **`protected`** und **`private`** Member
  - der geerbten Basisklasse `CAuto` in allen von `CAuto` abgeleiteten Klassen (`CCabrio`, `CBus`) erreichen.
  - in anderen Objekten der Basisklasse `CAuto` erreichen.
- Das Deklarieren von Member*variablen* als **`protected`** ist normalerweise ein Designfehler.
  - Problem der Softwarewartung, da für Codeänderungen *jede* Nutzung in allen abgeleiteten Klassen gefunden werden muss.
  - Membervariablen sollten **`private`** sein.
- Eine **`protected`** Schnittstelle sollte nur Funktionen, Typen und Konstanten enthalten.
- **`protected`** ist sinnvoll und üblich, um Member*funktionen* für Nutzung in abgeleiteten Klassen zu definieren.

0.3.5.9 Modifikatoren für Basisklassen bei Vererbung

Beispiel: public-Vererbung in C++

```
1 class CCabrio : public CAuto
2 {  /* ... */    };
```

Sichtbarkeit in Memberfunktionen der abgeleiteten Klasse

- `:public` Vererbung:
  - `public`: Member aus Basisklasse bleiben `public`:
  - `protected`: Member aus Basisklasse bleiben `protected`:
- `:protected` Vererbung:
  - `public`: Member aus Basisklasse werden `protected`:
  - `protected`: Member aus Basisklasse bleiben `protected`:
- `:private` Vererbung (Default):
  - `public`: Member aus Basisklasse werden `private`:
  - `protected`: Member aus Basisklasse werden `private`:
- Für alle Vererbungsarten gilt:
  - `private` Member aus Basisklasse sind nicht sichtbar

0.3.5.10 Modifikatoren für Basisklassen bei Vererbung (2)

Konvertierung von Zeigern und Referenzen

```
1 CAuto* myAuto = new CCabrio;
```

- Modifikatoren für Basisklassen kontrollieren Konvertierung von Zeigern und Referenzen abgeleiteter Klassentypen zum Typ der Basisklasse. Wichtig bei Polymorphie.
- `:public` Vererbung: Jede Funktion kann Zeiger `CCabrio*` in Zeiger `CAuto*` konvertieren.
- `:protected` Vererbung: Nur Memberfunktionen von `CCabrio` und der von `CCabrio` abgeleiteten Klassen können Zeiger `CCabrio*` in Zeiger `CAuto*` konvertieren.

© R.-R. Grigat 2003-2019

- `:private` Vererbung (Default): Nur Memberfunktionen von `CCabrio` können Zeiger `CCabrio*` in Zeiger `CAuto*` konvertieren.

Beispiel: KP23_2_protected_tresor, KP23_3_konto_Aufgabe, KP23_4_productFood_Aufgabe

0.3.5.11 Überschreiben von Funktionen

Definition: *Überschreiben* von Funktionen (engl. *override*)

- Veränderung der Funktionalität einer Methode in abgeleiteter Klasse
- Dazu in abgeleiteter Klasse Methode mit gleichem Namen, *gleicher* Signatur und gleichem Rückgabetyp, aber anderer Funktionalität definieren

zum Vergleich: *Überladen* von Funktionen (engl. *overload*)

- gleicher Funktionsname mit *unterschiedlicher* Signatur
- Funktionen können in derselben Klasse und in abgeleiteten Klassen überladen werden

0.3.5.12 Überschreiben von Funktionen in C++

Beispiel C++

```
1 //cauto.hpp
2 class CAuto
3 {
4   public:
5     virtual void lackiere(std::string farbe); // Tip: niemals nicht-virtuelle
6 /* ... */   };                  // Funktion überschreiben, siehe bei Polymorphie
```

```
1 //ccabrio.hpp
2 class CCabrio : public CAuto
3 {
4   public:
5     void lackiere(std::string farbe);
6 };  //ueberschreiben von lackiere
```

© R.-R. Grigat 2003-2019

```
1 //cabrio.cpp
2 void CCabrio::lackiere(string farbe)
3 {
4   cout << "Dach mit Folie schuetzen";
5   CAuto::lackiere(farbe);
6 }
```

- **virtual** wichtig bei Polymorphie (Abschnitt 0.3.7)
- *Achtung:* **virtual** dann auch bei Destruktoren nötig. Tip: Destruktor *immer* **virtual** deklarieren.
- **virtual** wird vererbt, gilt auch in abgeleiteten Klassen

0.3.5.13 Aufruf von Funktion der Basisklasse in C++ trotz Überschreibens

Beispiel C++

```
1 //cabrio.cpp
2 void CCabrio::lackiere(string farbe) //Ueberschreiben der Funktion
3 {                                     //der Basisklasse
4   cout << "Dach mit Folie schuetzen";
5   CAuto::lackiere(farbe);           //Aufruf der Funktion der Basisklasse
6 }
```

- `CCabrio::lackiere` ruft Funktion `CAuto::lackiere()` auf
- Spezifizierung der Basisklasse (auch bei Mehrfachvererbung) mit Scope-Operator `::`

Beispiel: KP25_1_virtual, KP25_2_virtualDestructor

0.3.5.14 Final

Finale Klasse, finale Methode

- Schlüsselwort `final` erzeugt eine *finale Klasse*, die nicht weiter abgeleitet werden kann.

© R.-R. Grigat 2003-2019

- Schlüsselwort `final` nach Deklaration einer virtuellen Memberfunktion *innerhalb* der Klassendefinition verhindert weiteres Überschreiben der Memberfunktion.
- `final` ist ein kontextuelles Schlüsselwort und besitzt je nach Kontext unterschiedliche Bedeutung. Das Unterbinden des Überschreibens ist nur am Ende der Deklaration der Memberfunktion innerhalb der Klassendefinition möglich.

```
1  class CCabrio final : public CAuto
2  {  /* ... */ };
3  class CBus : CAuto{
4  void waschen() const noexcept override final;
5  };
```

0.3.5.15 Override

Schlüsselwort Override

- Schlüsselwort `override` am Ende der Signatur einer überschriebenen Memberfunktion erzeugt eine Fehlermeldung, falls in der Basisklasse keine entsprechende virtuelle Methode mit gleichen Aufrufparametern und gleichem Rückgabetyp existiert.
- `override` steht grundsätzlich ganz am Ende der Deklaration der Memberfunktion innerhalb der Klassendefinition, allerdings vor dem Schlüsselwort `final`.

```
1  void waschen() override;
2  void lackiere(string) override final;
3  std::string getFarbe() const noexcept override final;
```

0.3.5.16 Abstrakte Klasse

Definition: Abstrakte Klasse

1. Abstrakte Klasse dient zur Deklaration einer Schnittstelle
2. Instanzen (d. h. Objekte) abstrakter Klassen sind nicht erlaubt
3. Memberfunktionen abstrakter Klassen dürfen aber definiert werden
4. In C++ wird Klasse abstrakt durch *rein virtuelle Memberfunktion*

© R.-R. Grigat 2003-2019

Abbildung 10: UML-Diagramm: abstrakte Klasse (Klassenname *kursiv*)

0.3.5.17 Rein virtuelle Funktionen und abstrakte Klassen in C++

Definition: Rein virtuelle Memberfunktion

- Durch =0 in Deklaration wird virtuelle Funktion zu *rein* virtueller Funktion
- Eine Klasse mit mindestens einer rein virtuellen Funktion ist abstrakte Klasse
- Abgeleitete Klasse ist ebenfalls abstrakt, falls rein virtuelle Funktion nicht überschrieben wird (stückweise Implementierung von Schnittstellen)
- Compiler verhindert Erzeugung von Objekt einer abstrakten Klasse

Beispiel C++

```
1    virtual void lackiere (std::string farbe) = 0;
```

Beispiel: KP26_1_abstrakteKlasse_Mitarbeiter

0.3.5.18 Ergänzungen - Abstrakte Klasse

- Es ist nicht möglich explizit eine Instanz einer abstrakten Klasse zu erzeugen.

```
1 CAuto* automobil = new CAuto(); // FEHLER!
```

- Eine rein virtuelle Funktion darf in der Basisklasse definiert werden. Definition und Deklaration muss in diesem Fall getrennt werden.

```
1 void CAuto::lackiere(){cout << "lackiere..." << endl;} // erlaubt
```

- Die Definition kann in der Tochterklasse mittels *scope* Operator '::' erreicht werden.

```
1 class CCabrio : public CAuto
2 {
3   public:
4     void lackiere(){CAuto::lackiere();}
5 };
```

© R.-R. Grigat 2003-2019

0.3.5.19 Ergänzungen (2) - Virtuelle Funktionen

- Eine (*nicht* rein) virtuelle Funktion muss in der Klasse, die sie als erste deklariert auch definiert werden.
- Eine rein virtuelle Funktion darf, muss aber nicht in der Basisklasse definiert werden.
- Eine rein virtuelle Funktion, die in der abgeleiteten Klasse nicht definiert wurde, bleibt rein virtuell. Deshalb ist die abgeleitete Klasse dann auch eine abstrakte Klasse.

Beispiel: KP26_2_inhomListe_abstrBasisklasse

© R.-R. Grigat 2003-2019

0.3.6 Befreundete Funktionen

Problem

- Einer einzelnen anderen Klasse soll Zugriff auf private Elemente der eigenen Klasse ermöglicht werden
- Zugriff von außerhalb eigener Klasse bei **`private`** grundsätzlich nicht erlaubt

Lösung

- Einzelne andere Klasse als **`friend`** der eigenen Klasse deklarieren
- Zugriff auf eigene private Objekte dann möglich

Achtung

- **`friend`** durchbricht Prinzip der Kapselung
- Mit Vorsicht einsetzen!

0.3.6.1 Beispiel zu `friend` in C++

Beispiel C++: Auto mit Hupe

```
1 //chupe.hpp
2 class CHupe
3 {
4   friend class CAuto; //CAuto Zugriff
5   private:          //ermoeglichen
6     CHupe(std::string ton,
7         int staerke); //Konstruktor
8     ~CHupe();              //private!
9     std::string m_ton;
10    int m_staerke;
11 };
```

```
1 //cauto.hpp
2 class CAuto
3 {
4   public:
5     CAuto();
```

© R.-R. Grigat 2003-2019

```
6     std::string farbe();
7     void lackiere(std::string farbe);
8     CHupe m_hupe;
9       //nur wegen friend erlaubt
10  private:
11    std::string m_farbe;
12 };
```

- Konstruktor CHupe ist **private**
- Objekt kann nur von CAuto erzeugt werden
- keine Hupe ohne Auto

© R.-R. Grigat 2003-2019

0.3.7 Polymorphie

0.3.7.1 Polymorphie in C++

Beispiel C++

```
1 CAuto *auto1 = new CCabrio;//Upcast
2 CAuto *auto2 = new CBus;  //Upcast
3
4 auto1->waschen(); // Cabrio waschen
5 auto2->waschen(); // Bus waschen
```

- **delete** nicht vergessen!

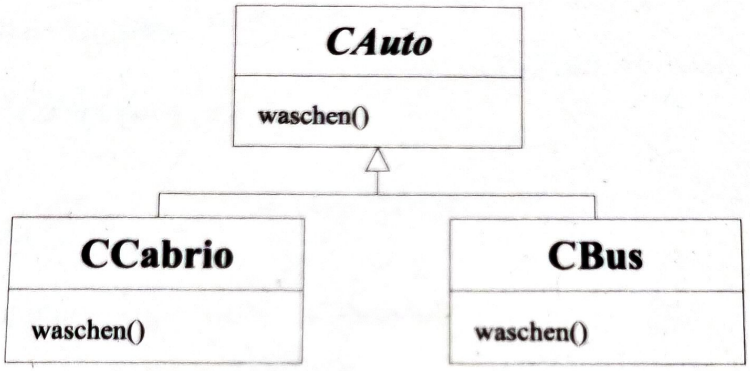


Abbildung 11: UML-Diagramm zu Polymorphie-Beispiel

Polymorphie:

- Zur Laufzeit (=„*späte Bindung*“) wird zum Objekt passende Version der Memberfunktion automatisch ausgewählt
  - Version aus Basisklasse oder überschriebene Version der zum Objekt gehörigen abgeleiteten Klasse
  - Späte Bindung muss in C++ durch **virtual** vor der Memberfunktion in Basisklasse erzwungen werden
  - In Java wird grundsätzlich immer späte Bindung verwendet
- Objekt wird als Zeiger (CAuto*) oder Referenz (CAuto&) auf Basisklasse gespeichert.

© R.-R. Grigat 2003-2019

- Ermöglicht weitere Trennung der Schnittstelle (`CAuto`) von der Implementierung (`CCabrio`, `CTaxi`, `CBus`)
- Destruktor der Basisklasse *muss* `virtual` deklariert sein, sonst wird nur Speicher der Basisklasse freigegeben
- Gefahr unerwarteter Nebeneffekte: In C++ *niemals* nicht-virtuelle Memberfunktionen überschreiben!

0.3.7.2 Polymorphie - virtual

- Eine Klasse wird als *polymorph* bezeichnet, wenn sie mindestens eine virtuelle Funktion besitzt.

```
1 class Auto
2 {
3   public:
4     virtual ~Auto();
5 };
```

- Nur für polymorphe Klassen wird die sogenannte *vtable* mit den überschriebenen Methoden-Varianten erzeugt.
- Bei der Erzeugung einer Instanz einer solchen Klasse wird ein (für den Programmierer unsichtbarer) Zeiger auf die zu dieser Klasse korrespondierende *vtable* abgelegt. Auf diese Weise kann zur Laufzeit der korrekte Einsprungspunkt für jede virtuelle Funktion gefunden werden.

0.3.7.3 Polymorphie - virtual (2)

- `virtual` wird vererbt, Schlüsselwort `virtual` muss beim Überschreiben nicht wiederholt werden
- Zum Überschreiben virtueller Funktion muss in Definition der abgeleiteten Klasse die neue Version deklariert (und nicht nur außerhalb definiert) werden, sonst wird Version aus Basisklasse verwendet.
- Sollen bei Aufruf überschriebener Memberfunktion Berechtigungen der Basisklasse (Schnittstelle) gelten, so macht man die überschriebene Version in abgeleiteter Klasse `private`. Dann sind Aufrufe der überschriebenen Version *nur* über späte Bindung möglich (Polymorphie, Zeiger/Referenz vom Typ der Basisklasse).

© R.-R. Grigat 2003-2019

0.3.7.4 Polymorphie - virtual (3)

Konstruktoren und Destruktoren bei Polymorphie

- Konstrukturen sind *niemals* `virtual`. Destruktoren sollten *immer* `virtual` sein wenn Klasse abgeleitet wird, damit bei Polymorphie auch Speicherbereich der abgeleiteten Klasse freigegeben wird.
- Konstruktoren und Destruktoren sollten keine virtuellen Funktionen aufrufen. In diesem Fall würde stets die Version der aktuellen Klasse (z.B. Basisklasse) aufgerufen werden. Grund ist, dass Konstruktoren in der Ableitungshierarchie top-down und Destruktoren bottom-up aufgerufen werden und die anderen Stufen noch nicht bzw. nicht mehr existieren.

0.3.7.5 Polymorphie - Upcasts

Definition: Upcast

Allgemein Typänderung (Cast) vom Typ einer abgeleiteten Klasse zum Typ ihrer Basisklasse

Anwendung Ermöglicht Entscheidung zur Laufzeit, welche Memberfunktion ausgeführt werden soll (RTTI Run Time Type Information)

In C++ Upcasts nur mit Zeigern oder Referenzen(C++) auf Klassen möglich

Im Beispiel Zeiger auf Objekt vom Typ der abgeleiteten Klasse (z. B. `CCabrio*`) wird in Variable vom Typ Zeiger auf Basisklasse (z. B. `CAuto*`) gespeichert

0.3.7.6 Polymorphie - Upcasts (2)

Upcast und Modifikator der Basisklasse

- Die Menge der Methoden, die einen Upcast von `CCabrio*` auf `CAuto*` ausführen dürfen, ist durch den Modifikator der Basisklasse `CAuto` eingeschränkt
- `class CCabrio : public CAuto \{...\};` Upcast darf in allen Methoden und Funktionen (einschließlich `main`) erfolgen
- `class CCabrio : protected CAuto \{...\};` Upcast darf nur in Methoden der abgeleiteten Klasse `CCabrio` und Methoden der davon weiter abgeleiteten Klassen erfolgen

© R.-R. Grigat 2003-2019

- `class CCabrio : private CAuto \{...\};` Upcast darf nur in Methoden der abgeleiteten Klasse `CCabrio` erfolgen

0.3.7.7 Statische Bindung, nicht-virtuelle Funktion

waschen nicht-virtuell

```
1  CAuto *auto1 = new CCabrio;//Upcast
2  CAuto *auto2 = new CBus;   //Upcast
3
4  auto1->waschen(); // Auto waschen
5  auto2->waschen(); // Auto waschen
```

Erreichbarkeit von Membervariablen und Memberfunktionen

- Ohne **`virtual`** (also nicht-virtuelle Funktion `waschen`) entscheidet der Zeigertyp über die Version der aufgerufenen Funktion (in den Zeilen 4 und 5 also Version der Basisklasse `CAuto`).
- Bei **`virtual`** (also virtuelle Funktion) entscheidet der Objekttyp über die Version der aufgerufenen Funktion (z.B. `CCabrio`), nicht der Typ des Zeigers (`CAuto*`).

0.3.7.8 Polymorphie - Upcasts (4)

Herausfinden des Objekttyps bei Downcast

- Herausfinden des Objekttyps durch Downcast mit **`dynamic_cast<>()`**, wie z.B. **`dynamic_cast`**`<CCabrio*>(auto1)`.
- Schlägt Downcast fehl, so ist Rückgabe der **`nullptr`**.

Niemals nicht-virtuelle Methoden überschreiben

- Bei Polymorphie würde die Funktionsversion des Basisklassenbereichs aktiviert, selbst wenn überschriebene Variante existiert.
- Da die aktivierte Funktion zur Basisklasse gehört kann diese alle privaten Elemente von Objekten der Basisklasse und in abgeleiteten Klassen alle privaten Elemente deren Basisklassenanteils erreichen.

Beispiel: KP25_3_dynamicCast

© R.-R. Grigat 2003-2019

0.3.7.9 Polymorphie - vtable

Beispiel vtable der Basisklasse:

```
1  // auto.cpp
2  class Auto
3  {
4    public:
5      virtual void waschen(){cout << "wasche Auto" << endl;}
6      // Pseudocode - KEIN C++!
7      // Initialisiere Zeiger d. Instanz auf vtable
8      Auto():_vptr(&Auto::_vtable[0]){};
9      FunctionPtr* _vptr;
10     // Erzeuge eine Liste von FunctionPtr
11     // der Länge 1 mit einem Zeiger auf
12     // die Funktion Auto::waschen
13     FunctionPtr _vtable[0] = {&Auto::waschen};
14 };
```

0.3.7.10 Polymorphie - vtable

Beispiel vtable einer Tochterklasse:

```
1  // cabrio.cpp
2  class Cabrio : public Auto
3  {
4    public:
5      void waschen(){cout << "wasche Cabrio" << endl;}
6      // Pseudocode - KEIN C++!
7      // Initialisiere Zeiger d. Instanz auf vtable
8      Cabrio():_vptr(&Cabrio::_vtable[0]){};
9      FunctionPtr* _vptr;
10     // Erzeuge eine Liste von FunctionPtr
11     // der Länge 1 mit einem Zeiger auf
12     // die Funktion Cabrio::waschen
13     FunctionPtr _vtable[0] = {&Cabrio::waschen};
14 };
```

0.3.7.11 Polymorphie - vtable

Unsichtbare Verwendung der vtable:

```
1 int main()
2 {
3   // _vptr wird auf die vtable von Auto gesetzt
4   Auto* automobil = new Auto();
5   // _vptr wird auf die vtable von Cabrio gesetzt
6   Auto* cabrio = new Cabrio();
7   // Wird übersetzt als automobil->_vptr[0], also Auto::waschen
8   automobil->waschen();
9   // Wird übersetzt als cabrio->_vptr[0], also Cabrio::waschen
10  cabrio->waschen();
11 }
```

Ausgabe:

```
1 wasche Auto
2 wasche Cabrio
```

Beispiel: KP30_3_sprungtabelleFunktionen

Übungsaufgabe C++ Klassenelemente

Die folgende Aufgabe gibt einen tieferen Einblick in Polymorphie und Vererbungsmechanismen. Diese Aufgabe thematisiert die Tücken der Polymorphie und muss zu diesem Zweck natürlich Code enthalten, den ein guter Programmierer niemals verwendet. Protected Variablen sind bis auf wenige Spezialfälle ein Designfehler, man sollte entweder kapseln (private) oder freigeben (public). Die vorliegende Aufgabe thematisiert also speziell protected und private Membervariablen und Memberfunktionen, denn public Member sind ja ohnehin überall sichtbar.

Eine polymorphe Basisklasse (d.h. eine Basisklasse mit virtueller Funktion) sollte immer einen virtuellen Destruktor enthalten, dieser ist in der Aufgabe weggelassen. Alle mit new erzeugten Speicherbereiche müssen mit delete wieder freigegeben werden. Auch dieses ist in der Aufgabe weggelassen.

Gegeben ist folgendes C++ Programm:

```
1  class CKfz
2  {
3  public:
4          virtual void beschleunige(CKfz& kfz)=0;
5          virtual void meldeBewegung(CKfz& kfz);
6  protected:
7          int geschwindigkeit_;
8          bool inBewegung_;
9  };
10
11 class CPkw : public CKfz
12 {
13 public:
14         void beschleunige(CKfz& kfz){kfz.geschwindigkeit_ = 80;};
15 };
16 class CMotorrad : public CKfz
17 {
18 public:
19         void beschleunige(CKfz& kfz){kfz.geschwindigkeit_ = 120;};
20 };
21
22 void CKfz::meldeBewegung(CKfz& kfz)
23 {
24         kfz.inBewegung_ = true;
25 }
26
27 int main(int argc, char *argv[])
28 {
29         CKfz* trabbi = new CPkw;
30         CKfz* chevi  = new CPkw;
31         CKfz* harley = new CMotorrad;
32
33         trabbi->inBewegung_ = true;
34         chevi->beschleunige(*trabbi);
35         trabbi->beschleunige(*harley);
36         harley->meldeBewegung(*chevi);
37 }
```

a) Definition des Modifikators **protected** von *Membervariablen* und *Memberfunktionen*?

b) Definition des Modifikators **protected** bei der *Ableitung von Klassen*?

c) Zeichnen Sie ein UML Diagramm (ohne Attribute und Operationen), das die Beziehung zwischen `CKfz`, `CPkw` und `CMotorrad` beschreibt.

d) Welche Bedingungen müssen für den Modifikator der Memberfunktion, den Modifikator der Membervariablen und für den Modifikator der Ableitung erfüllt werden, damit in Zeile 33 der Zugriff auf `trabbi->inBewegung_` zulässig ist? Ist dies erfüllt (kurze Begründung)?

e) Welche Bedingungen müssen für den Modifikator der Membervariablen und für den Modifikator der Ableitung erfüllt werden, damit in Zeile 34 die Ausführung von `chevi->beschleunige(*trabbi);` zulässig ist? Ist dies erfüllt (kurze Begründung)?

f) Ist die Anweisung in Zeile 36 erlaubt? Begründen Sie.

g) Was ändert sich für die Zeilen 33 bis 36, wenn die Zeile 11 geändert wird in `class CPkw : private CKfz`?

Lösung der Übungsaufgabe

Wichtig, bitte unternehmen Sie zunächst einen ernsthaften Lösungsversuch der Aufgabe und lesen erst danach hier weiter.

Nicht-virtuelle Funktionen sollten niemals überschrieben werden, denn bei Polymorphie wird dann die nicht-virtuelle Version der Basisklasse (entgegen der Erwartung) aktiviert und nicht (intuitiv erwartet) die überschriebene Variante der abgeleiteten Klasse.

Bei Polymorphie sind nur Elemente der Basisklasse erreichbar. Also falls ein (Zeiger auf ein) Objekt vom Type CPkw in einer Zeigervariable CKfz* gespeichert wird, dann sind nur die Elemente der Basisklasse des Objekts CPkw sichtbar. Ausnahme: Nur falls eine Funktion der Basisklasse CKfz virtuell ist (d.h. vtable angelegt wurde) und in der abgeleiteten Klasse CPkw überschrieben wurde, dann wird die überschriebene Version der Funktion aktiviert.

Nun zu den Aufgabenteilen

a) **protected** Membervariablen und **protected** Memberfunktionen sind nur in Methoden der Klasse selbst und in den Methoden der davon abgeleiteten Klasse sichtbar.

b) In **protected** abgeleiteten Klassen besitzen sowohl **public** als auch **protected** Membervariablen und Memberfunktionen der Basisklasse in der abgeleiteten Klasse die Eigenschaft **protected**. **private** Elemente der Basisklasse sind grundsätzlich nicht in Methoden von abgeleiteten Klassen oder in Methoden außerhalb der Klasse (z.B. main) sichtbar.

Darüber hinaus ist ein cast auf die Basisklasse nur innerhalb von den **protected** abgeleiteten Klassen erlaubt, nicht aber z.B. in der main-Funktion.

c) siehe Skript, offene Dreiecke verwenden!

d) Zugriff erfolgt aus `main`, dies ist keine Memberfunktion, also erfolgt ein Zugriff "von außen". Dazu muss die Membervariable `inBewegung_` und auch die Vererbung **public** sein. Die Membervariable ist aber **protected**, die Zugriffsbedingung ist also nicht erfüllt.

e) `chevi` vom Typ `CPkw*` ist auf `CKfz*` gespeichert. Wäre `beschleunige` in der Basisklasse nicht virtuell, dann würde beim Aufruf die Version der Basisklasse aktiviert werden, auch wenn `beschleunige` in `CPkw*` überschrieben wäre, und `beschleunige` besitzt während der Ausführung alle Rechte einer Methode der Klasse `CKfz*` und nicht `CPkw*` (daher niemals nicht-virtuelle Methoden überschreiben!). In der Aufgabe ist `beschleunige` aber virtuell, dadurch wird die v-Table angelegt und die überschriebene Version von `beschleunige` mit den Zugriffsrechten von `CPkw*` ausgeführt. `chevi` gilt in `beschleunige` also als `CPkw*`.

Dagegen wird `*trabbi` als `CKfz&` an `beschleunige` übergeben, also als anderer Name für ein schon vorhandenes Objekt (Referenz) vom Typ `CKfz`. Nun darf aber eine Memberfunktion beschleunige aus `CPkw` zwar auf die nicht-privaten Elemente seiner Basisklasse `CKfz` für alle Objekte vom Typ `CPkw` zugreifen, nicht aber auf **protected** oder **private** Elemente von Objekten der Basisklasse `CKfz` oder anderen davon abgeleiteten Klassen wie `CMotorrad`. In diesem Fall ist also `*trabbi` vom Typ `CKfz` nicht für `chevi` vom Typ `CPkw` sichtbar. Daher müsste `(*trabbi).geschwindigkeit_` **public** sein, dies ist nicht erfüllt und Zeile 34 ist nicht zulässig. Zusätzlich müsste die Ableitung **public** sein, das wäre hier gegeben.

Zusatzfrage:
Warum darf die Memberfunktion der Klasse `CPkw` nicht auf die Elemente ihrer Basisklasse zugreifen? Die Vererbung erfolgt **protected** und somit sollte die abgeleitete Klasse Zugriff auf diese Elemente ihrer Basisklasse haben, oder?

Antwort:
Das ist halt so definiert und ist tatsächlich wichtig zu wissen. Ein Objekt vom Typ `CKfz` ist etwas anderes als ein Objekt vom Typ `CPkw` (mit geerbten Eigenschaften

© R. Grigat 2003-2019

CKfz). Die Methoden des CPkw-Objekts können **protected** Member des CKfz Objekts nicht sehen und sie können ebenfalls nicht die **protected** Member der Basisklasse CKfz eines Objekts vom Typ CMotorrad sehen, denn beide Objekte haben nichts mit der Basisklasse von CPkw zu tun!

Subtil wird die ganze Geschichte, wenn Sie einen Zeiger auf ein Objekt vom Typ CPkw auf einen Zeiger vom Typ der Basisklasse CKfz speichern (Polymorphie). Jetzt wird es beim Aufruf der Methode aus CPkw wichtig, ob der Code der überschriebenen Version aus CPkw ausgeführt wird, oder der Code aus der Basisklasse CKfz. Wenn es der Code aus der Basisklasse CKfz ist, dann darf dieser auf **protected** Member von Objekten des Typs CKfz zugreifen und auch auf Member der Basisklasse von Objekten des Typs CMotorrad! Wenn es der Code aus der überschriebenen Version aus CPkw ist, dann darf er dies nicht. Der Code aus der Basisklasse wird z.B. aktiviert (und das ist der Fettnapf, den jeder C++ Programmierer kennen sollte), wenn die Methode in der Basisklasse CKfz nicht virtuell gemacht wurde. Es existiert dann keine vtable und der Compiler weiß bei Polymorphie daher nicht, dass es noch eine überschriebene Version in CPkw gibt, weil er bei Polymorphie grundsätzlich nicht an Membervariablen von CPkw herankommt und nur über die vtable an Memberfunktionen von CPkw.

Das ganze Problem tritt nicht auf, wenn Sie auf **protected** Variablen verzichten. Daher gelten **protected** Variablen als Designfehler.

f) meldeBewegung und die Vererbung müssen **public** sein, das ist erfüllt. meldeBewegung ist Methode der Basisklasse CKfz, sie ist nicht überschrieben und wird daher beim Aufruf aktiviert. Da meldeBewegung die Zugriffsrechte einer Methode der Basisklasse hat (und nicht die Zugriffsrechte der Klasse CMotorrad), darf sie auf alle Member der Basisklasse in Objekten der Basisklasse CKfz und in Objekten der von CKfz abgeleiteten Klassen zugreifen (hier CPkw und CMotorrad, aber nur Elemente der Basisklasse!).

Das Argument von `meldeBewegung` `chevi` wird als Referenz auf Basisklasse übergeben, also CKfz&, damit ist der `chevi` nun vom Typ Basisklasse und seine **protected** Membervariable `(*chevi).inBewegung_` aus der Basisklasse CKfz von CPkw ist für die Methode `meldeBewegung` sichtbar, weil die ausgeführte Version von `meldeBewegung` aus der Basisklasse CKfz von CPkw stammt.

Die Bedingung ist also erfüllt.

g) Die Zeilen 33 bis 35 sind nach wie vor verboten. Zeile 36 ist weiterhin erlaubt, da nur CPkw von privater Basisklasse betroffen ist, aber CMotorrad und CKfz relevant sind.

0.3.7.12 Objektorientierte Programmierung: Zusammenfassung

Grundkonzepte

- Trennung von Schnittstelle und Implementation (Kapselung)
- Wiederverwendung von Programmcode durch
  - Komposition („hat-ein“)
  - Vererbung („ist-ein“)
  - Polymorphie